

Software Engineering Project Report



Ocean Sweep

*Prepared by
Mohammed Abdul Hadi, Dev Shah, Jash Patel, Kirtan Patel*

Table of Contents

	List of Figures.....	7
	List of Tables	8
I	Project Description	9
1	Project Overview	9
2	The Purpose of the Project	9
2a	The User Business or Background of the Project Effort	9
2b	Goals of the Project	10
2c	Measurement	10
3	The Scope of the Work	10
3a	The Current Situation	11
3b	The Context of the Work	11
3c	Work Partitioning	12
3d	Competing Products	12
4	The Scope of the Product	13
4a	Scenario Diagram(s).....	15
4b	Product Scenario List	15
4c	Individual Product Scenarios	16
5	Stakeholders	17
5a	The Client.....	17
5b	The Customer	18
5c	Hands-On Users of the Product.....	18
5d	Maintenance Users and Service Technicians	18
5e	Other Stakeholders	18
5f	User Participation.....	19
5g	Priorities Assigned to Users	19
6	Mandated Constraints	20
6a	Solution Constraints	20
6b	Implementation Environment of the Current System	20
6c	Partner or Collaborative Applications	21
6d	Off-the-Shelf Software	22
6e	Anticipated Workplace Environment.....	23
6f	Schedule Constraints.....	24
6g	Budget Constraints	25
7	Naming Conventions and Definitions	25
7a	Definitions of Key Terms.....	25
7b	UML and Other Notation Used in This Document	26
7c	Data Dictionary for Any Included Models.....	27

8	Relevant Facts and Assumptions	29
8a	Facts	29
8b	Assumptions	30
II	Requirements	31
9	Product Use Cases	31
9a	Use Case Diagrams	31
9b	Product Use Case List	34
9c	Individual Product Use Cases	35
10	Functional Requirements	39
11	Data Requirements	41
12	Performance Requirements	43
12a	Speed and Latency Requirements	43
12b	Precision or Accuracy Requirements	45
12c	Capacity Requirements	47
13	Dependability Requirements	48
13a	Reliability Requirements	48
13b	Availability Requirements	50
13c	Robustness or Fault-Tolerance Requirements	51
13d	Safety-Critical Requirements	51
14	Maintainability and Supportability Requirements	52
14a	Maintenance Requirements	52
14b	Supportability Requirements	53
14c	Adaptability Requirements	53
14d	Scalability or Extensibility Requirements	54
14e	Longevity Requirements	54
15	Security Requirements	55
15a	Access Requirements	55
15b	Integrity Requirements	55
15c	Privacy Requirements	55
15d	Audit Requirements	56
15e	Immunity Requirements	56
16	Usability and Humanity Requirements	56
16a	Ease of Use Requirements	56
16b	Personalization and Internationalization Requirements	56
16c	Learning Requirements	57
16d	Understandability and Politeness Requirements	57
16e	Accessibility Requirements	57

16f	User Documentation Requirements	57
16g	Training Requirements	58
17	Look and Feel Requirements	58
17a	Appearance Requirements	58
17b	Style Requirements	58
18	Operational and Environmental Requirements	59
18a	Expected Physical Environment	59
18b	Requirements for Interfacing with Adjacent Systems	59
18c	Productization Requirements	59
18d	Release Requirements	59
19	Cultural and Political Requirements	60
19a	Cultural Requirements	60
19b	Political Requirements	60
20	Legal Requirements	60
20a	Compliance Requirements	60
20b	Standards Requirements	60
20c	Requirements – Test Correspondence Summary	60
20d	Acceptance Test Descriptions	71
21	Design Goals	83
22	Current System Design	83
23	Proposed System Design	84
23a	Initial System Analysis and Class Identification	84
23b	Dynamic Modelling of Use-Cases	86
23c	Proposed System Architecture	86
23d	Initial Subsystem Decomposition	87
24	Additional Design Considerations	88
24a	Hardware / Software Mapping	89
24b	Persistent Data Management	91
24c	Access Control and Security	92
24d	Global Software Control	95
24e	Boundary Conditions	96
24f	User Interface	98
24g	Application of Design Patterns	78
25	Final System Design	99
26	Object Design	102
26a	Packages	102

	26b	Subsystem I	79
	26c	Subsystem II.....	79
	26d	etc.	105
III		Project Issues	108
27		Open Issues.....	108
28		Off-the-Shelf Solutions	109
	28a	Ready-Made Products	109
	28b	Reusable Components	110
	28c	Products That Can Be Copied	110
29		New Problems	111
	29a	Effects on the Current Environment.....	111
	29b	Effects on the Installed Systems.....	111
	29c	Potential User Problems	111
	29d	Limitations in the Anticipated Implementation Environment That May Inhibit the New Product	111
	29e	Follow-Up Problems	112
30		Migration to the New Product	112
	30a	Requirements for Migration to the New Product	112
	30b	Data That Has to Be Modified or Translated for the New System	113
31		Risks	113
32		Costs	116
33		Waiting Room	118
34		Ideas for Solutions	119
35		Project Retrospective.....	121
IV		Glossary	122
V		References / Bibliography	124
VI		Index	125

List of Figures

Figure 1- Context of Work.....	11
Figure 2-Scenario Diagram of Ocean Sweep.....	15
Figure 3-Use Case Diagram of Reporting trash location.....	32
Figure 4-Use Case Diagram of Volunteer Registration.....	32
Figure 5-Use Case Diagram of Organizing CleanUp Events.....	33
Figure 6-Use Case Diagram of Waster Reporting Coordination.....	33
Figure 7-Notification System.....	34

List of Tables

Table 1- Work Partitioning.....	12
Table 2- Product Scenario List.....	16
Table 3- User Profile.....	27
Table 4- Waste Report.....	28
Table 5- Clean Up Report.....	29
Table 6- Product Use Case List.....	34
Table 7- Requirements - Acceptance Tests Correspondence.....	60

I Project Description

1 Project Overview

OceanSweep is a web-based platform designed to address the growing issue of ocean and beach pollution by mobilizing community-driven efforts. The app connects divers, environmental groups, and volunteers to actively report and clean up waste, by using real-time mapping and notifications. With tools for waste tracking, event coordination, and community engagement, OceanSweep helps individuals and organizations to make a positive impact on marine conservation. The platform also provides educational resources to raise awareness about the dangers of plastic pollution and offers a recognition system to encourage long-term participation.

2 The Purpose of the Project

OceanSweep is being developed to address the issue of ocean and beach pollution, particularly the growing volume of plastic waste that harms marine life, ecosystems, and coastal communities. With 8 million tons of plastic entering the oceans annually, this project aims to provide a scalable, community-driven solution that helps volunteers, divers, and environmental organizations to actively reduce pollution.

The platform allows users to report waste sightings, track pollution through interactive maps, and coordinate cleanup events in real time. By fostering collaboration between individuals and groups, **OceanSweep** seeks to streamline efforts for more efficient waste removal and create a global network of environmentally-conscious participants. Additionally, the app offers educational resources to promote understanding of the environmental impacts of pollution, encouraging users to take a proactive role in conservation. The overarching goal is to significantly reduce oceanic and coastal waste, protect marine ecosystems, and promote sustainable habits within communities.

2a The User Business or Background of the Project Effort

The client's business for the OceanSweep project centers around environmental conservation and community-driven efforts to reduce ocean and beach pollution. The client operates within the broader environmental sector, with a focus on protecting marine ecosystems and reducing the negative impacts of plastic and other waste on oceans, beaches, and coastal communities. Their "business" involves not only environmental advocacy but also facilitating actionable community engagement through technology.

The client's primary aim is to participate in waste cleanup efforts and contribute to marine conservation. This aligns with the project's goal of providing tools for reporting waste, organizing cleanup events, and educating users on the effects of pollution. Therefore, the client's business is focused on helping communities to make a tangible impact on the environment through direct action and awareness-building, leveraging digital platforms to coordinate these efforts at scale.

2b Goals of the Project

The goal of the project is to help reduce pollution of the oceans. Pollution in oceans is not an individual or an organizational problem rather it is a pressing problem for humanity. By removing and reducing plastics and other non-biodegradable wastes from the oceans, the world would be much safer place for us to stay and sustainable as well.

Furthermore, OceanSweep project is being carried out to provide a practical and impactful solution to the pressing issue of ocean and beach pollution. The goal is to help individuals, environmental organizations, and volunteers to take direct and efficient action against marine waste, which threatens the health of ecosystems, wildlife, and coastal communities.

2c Measurement

When there is a potential and measurable drop in the amount of pollution in oceans and the world has adopted OceanSweep project method to reduce the amount of plastics and harmful materials, we could say that it is a step in the right direction and we are moving towards our goals.

Another measurable outcome would be the total volume of waste collected during the organized cleanups, providing concrete evidence of the project's impact on reducing pollution. Growth in the number of active users on the platform, including new registrations and participation in forums, would reflect a vibrant and engaged community committed to environmental protection.

Furthermore, an increase in the utilization of educational resources available within the app would showcase the project's effectiveness in raising awareness about marine conservation. Positive feedback from users and community organizations regarding their experiences and perceived improvements in local environmental conditions would also serve as a strong indicator that the project has met its objectives. Collectively, these outcomes will demonstrate that OceanSweep has successfully mobilized community efforts and contributed to the reduction of ocean and beach pollution.

3 The Scope of the Work

The work addressed by the OceanSweep project focuses on developing a platform that enables users to report waste sightings, coordinate cleanup events, and engage in community discussions about marine conservation. This specific subset of activities aims to streamline and enhance the efforts of individuals and organizations dedicated to reducing ocean and beach pollution, facilitating direct action and collaboration among community members.

3a The Current Situation

The current situation is not good. Every year, millions of tons of garbage end up in the oceans, with 80% originating from land-based sources and 20% from marine sources. Approximately 8 million pieces of plastic pollution find their way into the ocean daily, posing severe threats to marine life and ecosystems. Furthermore, the United Nations Environment Programme estimates that the global community suffers over \$8 billion in damage to marine ecosystems annually due to plastic pollution alone.

3b The Context of the Work

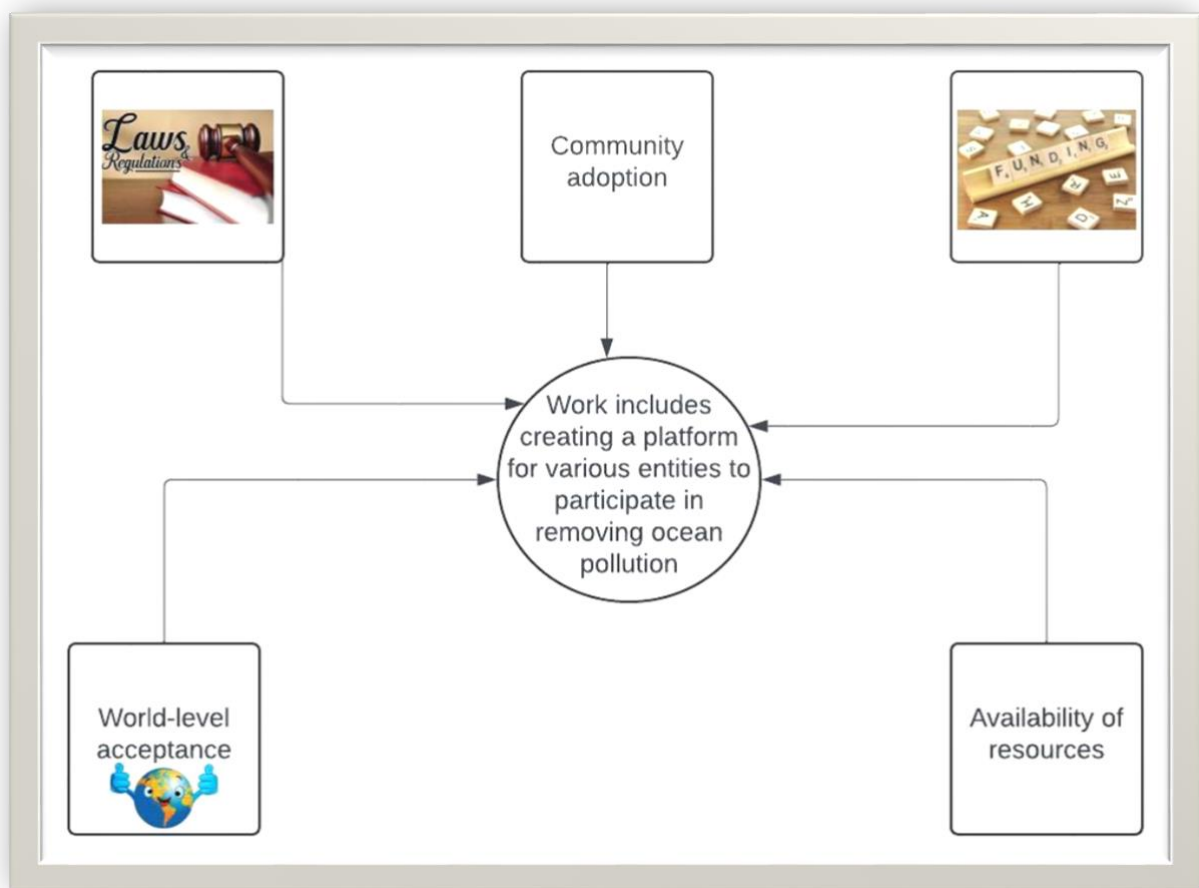


Figure 1 Context of work

3c Work Partitioning

Table 1 Work Partitioning

Business Event	Description	Response
Waste Report Submitted	A user reports a sighting of waste in the ocean or on a beach.	-Log the report in the database. -Notify nearby users and environmental groups. -Update the interactive map to reflect the new report.
Cleanup Event Created	An organization or user schedules a cleanup event.	-Add the event to the platform calendar. -Send notifications to users about the upcoming event. -Allow users to sign up and indicate any skills or equipment they can provide.
User Signs Up for an Event	A user expresses interest in joining a scheduled cleanup.	-Update the event's participant list. -Send confirmation to the user. -Provide reminders as the event date approaches.
Cleanup Event Completed	A scheduled cleanup event has taken place.	-Collect data on the amount of waste collected. - Update user profiles with participation details. -Send thank-you messages to participants and request feedback.
Community Forum Post Created	A user initiates a discussion or shares information in the forum.	-Log the post and notify users of new discussions. -Facilitate user engagement through replies and interactions. -Highlight important topics or tips for marine conservation.
Educational Resource Accessed	A user views or interacts with an educational module or resource.	-Track engagement metrics for each resource. -Suggest related content based on user interests. -Encourage users to share insights or takeaways in the community forum.
User Earns a Badge	A user completes certain milestones or contributions on the platform.	-Update the user profile to reflect new badges. -Send notification congratulating the user. -Highlight user achievements within the community to encourage further participation.

3d Competing Products

Two existing products address marine conservation and waste management, such as:

1. Marine Debris Tracker: This mobile app allows users to report and map debris in waterways. However, it lacks robust community engagement features.

2. Clean Swell: Developed by the Ocean Conservancy, this app focuses on tracking beach cleanup activities but primarily targets organized events without facilitating ongoing community discussions.

Despite these existing options, OceanSweep is still necessary for several reasons:

Holistic Approach: OceanSweep combines waste reporting, cleanup event coordination, and educational resources into a single platform.

Enhanced Community Engagement: The platform fosters a sense of community by providing tools for users to connect and plan events.

User Recognition and Motivation: With user profiles and badges, OceanSweep motivates participants and recognizes their contributions, encouraging sustained involvement.

In summary, while other products offer valuable features, OceanSweep's unified platform enhances community participation and empowers users to take meaningful action against marine pollution.

4 The Scope of the Product

Proposed Product Scenarios for OceanSweep

The OceanSweep platform focuses on mobilizing community efforts to combat marine and beach pollution through a user-friendly web application. It encompasses waste reporting, cleanup event coordination, community discussions, and educational resources. The following scenarios illustrate how different users interact with the platform in practical situations, highlighting the key features and functionalities included in the product.

1. Scenario: Reporting Marine Waste

- **User:** A diver named Alex discovers plastic debris while diving.
- **Action:** Alex opens the OceanSweep app, navigates to the interactive map, and reports the sighting by providing details on the type and amount of waste. The app sends real-time notifications to nearby environmental groups about the report.
- **Outcome:** Alex's report is logged, and local cleanup groups are alerted to take action.

2. Scenario: Organizing a Cleanup Event

- **User:** A local environmental group leader, Sarah, wants to organize a beach cleanup.

- Action: Sarah creates a cleanup event in the OceanSweep app, filling in details such as date, time, location, and any equipment needed. The event appears on the platform's calendar and map.
- Outcome: Community members receive notifications about the event, and volunteers can sign up to participate directly through the app.

3. Scenario: Participating in a Cleanup

- User: A college student named Jordan sees a notification about a nearby cleanup event.
- Action: Jordan clicks on the event details, sees the equipment needed, and signs up to participate. The app reminds him of the event as the date approaches.
- Outcome: Jordan attends the cleanup, where he meets other volunteers and contributes to collecting waste.

4. Scenario: Engaging in the Community Forum

- User: A concerned citizen, Maria, wants to learn more about marine conservation.
- Action: Maria visits the community forum on the OceanSweep app, where she reads discussions about effective cleanup strategies and shares her thoughts on the impact of pollution.
- Outcome: Maria connects with other users, gaining insights and tips while fostering a collaborative community atmosphere.

5. Scenario: Accessing Educational Resources

- User: A teacher wants to educate her students about ocean conservation.
- Action: She accesses the educational modules in the OceanSweep app, which includes articles and videos about marine pollution and conservation practices.
- Outcome: The teacher uses these resources in her classroom, empowering students with knowledge and encouraging them to participate in local cleanup efforts.

These scenarios demonstrate how OceanSweep addresses the critical work of connecting individuals, organizing efforts, and promoting education about marine conservation, ultimately enhancing community involvement in combating ocean and beach pollution.

4a Scenario Diagram(s)

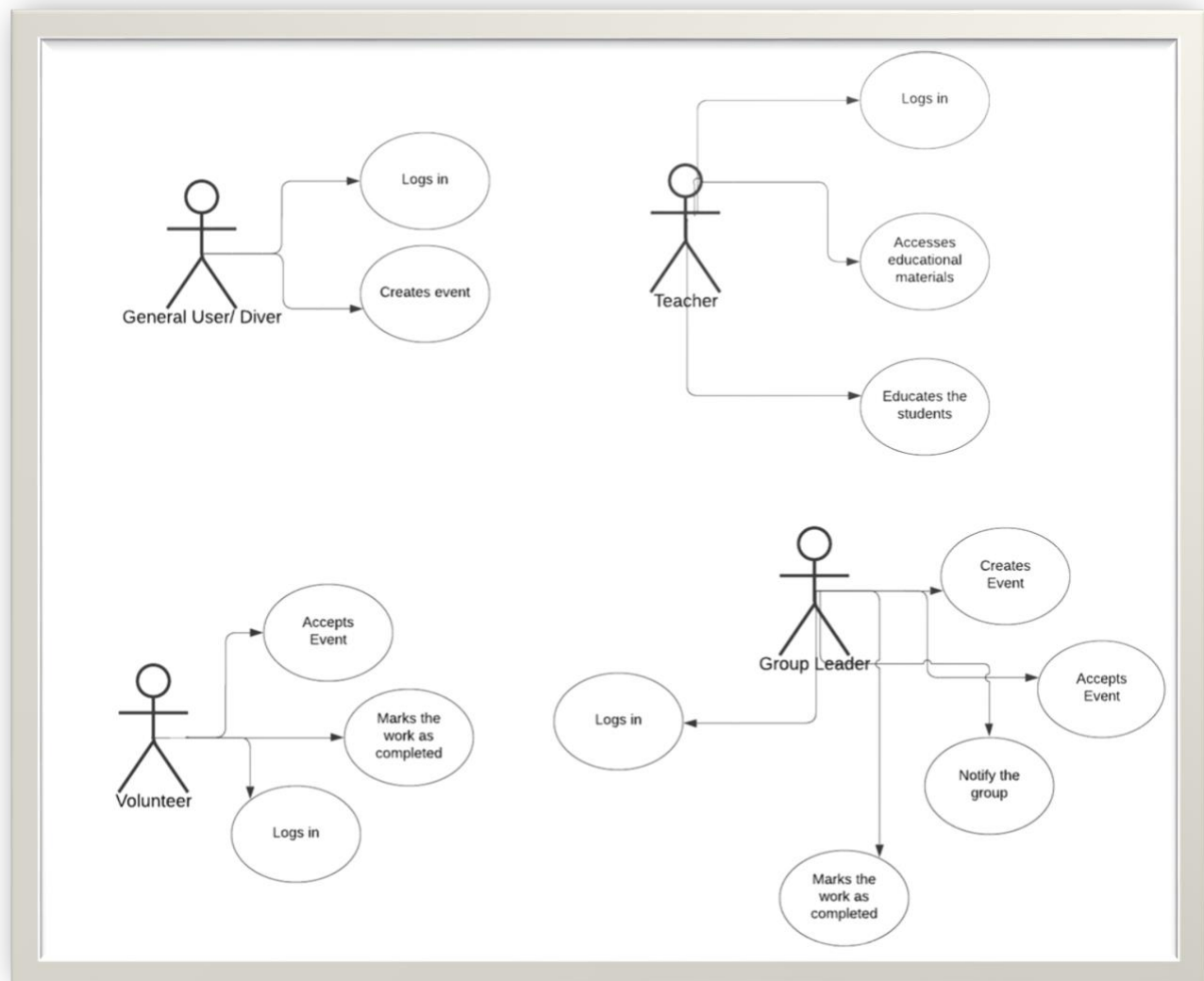


Figure 2 Scenario Diagram of OceanSweep

4b Product Scenario List

Table 2 Product Scenario List

Scenario Name	External Actors involved	Description
Report Marine Waste	Diver, Environmental Groups	Users can report sightings of marine waste using an interactive map. Each report includes details about the type and amount of waste.

Organize Cleanup Event	Group Leader	Group leaders can create and manage cleanup events, providing details on when and where they will occur.
Participate in Cleanup	Volunteer	Volunteers can sign up for cleanup events and indicate any special skills or equipment they can contribute.
Engage in Community Forum	Concerned Citizen	Users participate in discussions on topics related to marine pollution and conservation, sharing insights and tips.
Access Educational Resources	Teacher	Users can access learning modules and materials provided by environmental experts to better understand marine conservation.

4c Individual Product Scenarios

The following scenarios describe the practical interactions users will have with the OceanSweep platform. Each scenario outlines a specific use case and highlights the roles of the external actors involved.

Scenario 1: Reporting Marine Waste

Description: A diver, Alex, observes plastic debris while diving and reports it through the OceanSweep app. He uses the interactive map to pinpoint the location and specify the type and amount of waste. Once submitted, environmental groups in the vicinity receive real-time alerts to take action.

Scenario 2: Organizing a Cleanup Event

Description: Sarah, the leader of an environmental group, creates a cleanup event using the OceanSweep platform. She fills out details like the event's date, time, and location, and publishes it on the interactive map. Volunteers nearby can see the event and sign up to participate.

Scenario 3: Participating in a Cleanup

Description: Jordan, a volunteer, receives a notification about a cleanup event through the OceanSweep app. Interested in participating, he reviews the event details and signs up. He brings his own supplies and joins other volunteers at the designated location to help clean up.

Scenario 4: Engaging in the Community Forum

Description: Maria, a concerned citizen, accesses the community forum on OceanSweep to discuss marine conservation topics. She reads existing threads and posts a question about effective waste removal strategies, engaging with other users who share their insights and experiences.

Scenario 5: Accessing Educational Resources

Description: A teacher accesses the OceanSweep app to find educational resources on marine conservation. She explores learning modules and articles provided by environmental experts, integrating this information into her lesson plans for students.

5 Stakeholders

The OceanSweep project involves various stakeholders with distinct interests in its development and implementation:

Diver Community: Individuals who dive and are impacted by marine pollution, interested in reporting waste and participating in cleanup efforts.

Environmental Groups: Non-profit organizations focused on marine conservation, aiming to organize events and track their impact through the platform.

Volunteers: Community members eager to engage in cleanup activities, seeking opportunities to contribute and connect with others.

Local Government Authorities: Agencies responsible for environmental regulations, interested in collaborating on initiatives and monitoring improvements.

Educators and Students: Teachers and learners involved in marine science education, looking for resources to enhance their understanding of conservation.

Developers and Technical Team: The team designing and maintaining the platform, focused on creating an effective and user-friendly tool.

Community Leaders: Advocates for environmental initiatives, seeking to promote the platform and encourage community participation.

Funding Organizations: Entities providing financial support, interested in ensuring their funding leads to measurable results in reducing marine pollution.

5a The Client

For the OceanSweep project, the client consists of a collective group that includes environmental organizations, community leaders, and volunteers. This group plays a crucial role in funding the development of the platform and providing essential input throughout the process. By paying upfront for the product's development, they ensure that it aligns with their goals of addressing marine pollution and promoting community

engagement in conservation efforts. Their guidance and insights will help shape the features and functionality of the OceanSweep app, ensuring it effectively meets the needs of its users and contributes to meaningful environmental impact.

5b The Customer

In the case of the OceanSweep project, the customer encompasses various entities that will utilize the platform after its completion. This includes environmental organizations, community groups, and volunteers who will benefit from the app's functionalities. These customers are interested in leveraging the tool to report marine waste, coordinate cleanup events, and access educational resources. Since the platform is designed to facilitate community involvement in marine conservation, the customers will play an active role in its adoption and use, ensuring the product serves its intended purpose effectively. There is no external customer in this case, as the stakeholders and users themselves represent the customer base, promoting a collaborative effort toward improving ocean health.

5c Hands-On Users of the Product

The OceanSweep platform will be utilized by several key user groups:

1. Divers: They will report marine waste sightings and engage in cleanup efforts through the app.
2. Volunteers: Community members who will find and sign up for cleanup events organized via the platform.
3. Environmental Group Members: Individuals from non-profit organizations coordinating cleanup activities and tracking their impact.
4. Educators and Students: Teachers and learners accessing educational resources to enhance their understanding of marine conservation.
5. Community Advocates: Local leaders promoting events and encouraging community involvement in conservation efforts.

These users are essential for the platform's success, as their interactions will drive effective action against marine pollution.

5d Maintenance Users and Service Technicians

The maintenance users and service technicians would be just the developers who develop the OceanSweep project who would frequently update the software as well as debug any issues.

5e Other Stakeholders

Beyond the primary stakeholders, the OceanSweep project involves other parties with varying interests:

1. Local Business Owners: Businesses near coastal areas may support conservation but could object if cleanup activities disrupt operations.
2. Fishermen: Local fishing communities might appreciate pollution reduction but may have concerns about the impact of cleanup efforts on their fishing practices.
3. Government Regulators: Agencies responsible for environmental policies may need to adapt regulations based on data from the platform, leading to mixed reactions.
4. Skeptics: Individuals who doubt the effectiveness of community-driven initiatives may resist participation and support.
5. Research Institutions: Academics interested in data analysis may have concerns about the accuracy and effectiveness of the app in generating reliable insights.
6. Funding Entities: Organizations funding marine conservation may seek clear success metrics, placing pressure on the project to demonstrate measurable outcomes.

Engaging these stakeholders is essential to address concerns and foster support for the OceanSweep initiative.

5f User Participation

User participation in the development of the OceanSweep platform is expected to be significant and will include:

1. Feedback and Input: Users, such as divers and volunteers, will provide insights on initial concepts and prototypes to help shape the app's features.
2. Usability Testing: Selected users will test the app's interface, allowing developers to identify issues and improve the user experience.
3. Focus Groups: Organized discussions with diverse user representatives will gather in-depth feedback on objectives and functionalities.
4. Community Engagement: Ongoing surveys and forums will encourage users to share thoughts and suggestions throughout the development process.
5. Pilot Programs: Testing in select communities will provide real-world feedback for final adjustments before the official launch.

This active involvement will ensure that the OceanSweep platform effectively meets user needs and fosters community engagement in marine conservation.

5g Priorities Assigned to Users

In the OceanSweep project, certain user groups will hold higher priority due to their critical roles and contributions:

1. Environmental Group Members: These users are essential as they will lead cleanup initiatives and coordinate community efforts. Their feedback on the platform's features is vital for ensuring it meets organizational needs.
2. Volunteers: Engaging this group is crucial since they will actively participate in cleanup events. Their experiences and suggestions will directly influence user engagement and event coordination features.
3. Divers: While important for reporting marine waste and participating in cleanups, their involvement may be secondary to the primary organizers. However, their insights on specific diving-related features will still be valuable.
4. Educators and Students: This group will benefit from the educational resources offered by the platform. Their feedback will help shape content but is not as critical as input from those directly involved in cleanup activities.
5. Community Advocates: While they play a supportive role in promoting events, their priority is lower compared to users who will directly interact with the app for cleanup coordination.

By focusing on the needs and feedback of these prioritized user groups, the OceanSweep project can ensure that the platform is effective in fostering community engagement and addressing marine pollution.

6 Mandated Constraints

6a Solution Constraints

1. Open-source technology should be used in Ocean Sweep's development to cut costs and increase transparency. Standardized frameworks and protocols can be used to guarantee maintenance-free compatibility.
2. To guarantee broad accessibility for all users, including divers, volunteers, environmental groups, educators, and community members, the application must be a web-based platform that can be accessed through standard internet browsers.
3. Users must be able to view cleanup event locations and accurately report rubbish sightings using real-time interactive mapping tools on the site. The system needs to notify users in real time about newly reported waste and impending cleanup events to enable prompt responses.

6b Implementation Environment of the Current System

1. Hardware Environment:
 - User Devices: Standard desktop computers with operating systems like Windows, macOS, or Linux will be able to access the platform. Mobile web browsers for iOS or Android-powered smartphones and tablets will provide complete access to the platform.

- **Server Infrastructure:** To guarantee scalability and availability, OceanSweep will be hosted on dependable cloud infrastructure offered by providers such as Microsoft Azure, Google Cloud Platform (GCP), or Amazon Web Services (AWS). To manage fluctuating loads, the server environment has to provide auto-scaling, particularly during periods of high usage, such as global cleanup events.

2. Software Environment

- **Operating Systems:** The client side of the project would be compatible with the major operating systems such as Windows, macOS, Linux, iOS, and Android. For the project's server side of the project an operating system based on Linux that is reliable and secure, such as Ubuntu Server or CentOS, will power the backend.
- **Database Systems:** Enforcement of strict data security protocols to protect sensitive information. Implementation of a robust database system like PostgreSQL or MySQL to store user data, waste reports, event information, and educational resources.

3. Networks and Communications

- **User Access:** Users will need an internet connection to interact with the platform. The application will be optimized to function effectively even with limited bandwidth to accommodate users in areas with poor connectivity. Where possible, the platform will allow users to input data offline and synchronize when connectivity is restored, ensuring continuous engagement.
- **API's and Integration:** Integration with reliable mapping APIs like Google Maps or OpenStreetMap to provide real-time geolocation features for waste reporting and event coordination. Also, usage of web push notifications and email services to deliver real-time updates and reminders to users.

4. Security Environment

- **Data Encryption:** While the data is in transit, all data transmitted between the client and server will be encrypted using Secure Sockets Layer(SSL), Transport Layer Security(TSL) protocols, or similar protocols. While data is not in transit, all the sensitive data will be encrypted to prevent unauthorized access.
- **Authentication and Authorization:** Implementation of secure authentication mechanisms using Firebase while also implementing role-based access control such that every user would have appropriate access levels to ensure privacy and security.

6c Partner or Collaborative Applications

In developing Oceansweep, compatibility with various external applications and services is essential to enhance the platform's functionality and user experience:

1. To provide real-time interactive mapping features that enable users to report waste sightings accurately and view cleanup event locations, we will integrate mapping and geolocation services that allow for geolocation functionalities, route planning to event sites, and visual representation of waste reports on maps (Google Map API or OpenStreetMap API).
2. To send confirmation emails, newsletters, and updates to users regarding their activities on the platform, we will integrate some services like SendGrid, which will facilitate communication between the platform and users for event reminders, account verification, and notifications. Along with that, integration of web push notification services to deliver real-time updates and reminders that keeps the users engaged and informed without requiring them to actively keep browsing the platform.
3. To increase outreach and community engagement by allowing users to share waste reports, cleanup events, and achievements directly from the platform, integration of social media platforms is also a must. With that, users can log in using their social media accounts, simplifying the registration process and enhancing user experience.
4. To monitor user engagement, platform usage statistics, and gather insights to improve functionality and user experience, Google Analytics helps in understanding user behavior, identifying popular features, and making data-driven decisions for platform enhancements.
5. To support multiple languages and make the platform accessible to a global audience, Google Translate API facilitates the localization of content and user interfaces to accommodate users from different linguistic backgrounds.

6d Off-the-Shelf Software

The inclusion of certain commercial off-the-shelf (COTS) software is essential to enhance functionality, ensure reliability, and expedite development for Oceansweep.

1. The paid version of Google Maps API offers higher usage limits, better performance, and access to premium features not available in the free version, ensuring scalability as the user base grows; with the paid version of the API, Oceansweep can provide advanced mapping features, high-resolution satellite imagery, and real-time geolocation services essential for accurate waste reporting and event coordination.
2. Amazon Web Services(AWS) offers a comprehensive suite of cloud services with robust infrastructure, ensuring the platform can handle increasing loads and store data securely; with the help of AWS, we can host the application's servers and databases with high availability, scalability, and security.
3. To securely process payments for donations or fundraising activities, the project would require something similar to Stripe. It can provide a developer-friendly API

with strong security measures and support various payment methods, making it ideal for handling financial transactions.

4. The enterprise version of Google Analytics provides advanced analysis tools, data integration capabilities, and higher data processing limits necessary for data-driven decision-making, which can be used to gain in-depth insights into user behavior, engagement metrics, and platform performance.
5. Firebase Authentication is also an important integration with the project since it's various sign-in methods (email/password, social media logins) simplify the authentication process and enhance security.

6e Anticipated Workplace Environment

The OceanSweep platform is intended for usage by a broad range of users, such as volunteers, educators, community advocates, environmental group members, and divers. The expected work environments in which the platform will be used include a variety of contexts with particular human characteristics to take into account:

- Divers and boat operators may use the platform on or near the water, where moisture, splashes, and movement can affect device handling. Users will also often access the platform while participating in cleanup events on beaches or coastal regions. These environments can present challenges such as bright sunlight, glare, sand, and exposure to the elements.
- Design considerations to make the app better: For the platform to guarantee screen readability in bright sunlight, it must provide a user interface with high contrast images and movable brightness settings. Large icons and touch-friendly controls can make using a device easier for consumers who might be wearing gloves. Quick and simple interactions should be supported by the program to reduce the amount of time users must manage their devices in potentially dangerous situations. In places where connectivity is erratic, offline capabilities are advantageous. The platform ought to be low-bandwidth optimized and equipped with capabilities that facilitate offline data entry and access. When connectivity is restored, data synchronization ought to happen immediately to guarantee ongoing interaction without data loss.
- During cleanup operations, users may be exposed to wind, rain, or extremely high or low temperatures. Users may encounter noisy and distracting environments when participating in group cleanups or instructional programs. While on the go, a lot of people will rely on their smartphones or tablets. People with a range of skills, such as those with visual, auditory, or motor impairments, will use the platform.
- Design consideration to make the app better: It's imperative to have a responsive design that works with different screen sizes. The user interface must be simple to use and touchscreen-friendly, with streamlined navigation, to improve performance on mobile networks, the platform should reduce load times and

data consumption. The application should facilitate rapid interactions to minimize the amount of time spent under severe conditions when using with damp hands or when fine motor skills are impaired; button sizes and spacing should be accommodated. The platform's layout should be simple and uncomplicated for ease of use and comprehension. To consider noisy surroundings, audio feedback must be customizable or optional, it is necessary to adhere to the Web Content Accessibility Guidelines (WCAG) to make the platform more user-friendly, features like keyboard navigation, screen reader compatibility, movable text sizes, and captions for multimedia should be added to facilitate all users.

6f Schedule Constraints

The development and implementation of the OceanSweep platform are subject to several schedule constraints that must be considered to ensure timely delivery and maximum impact. These constraints are influenced by various factors:

- The project is planned to be completed within an eight-month timeframe to address the urgent issue of ocean and beach pollution effectively, where development will occur in phases, prioritizing core functionalities such as waste reporting and event coordination for early release to facilitate immediate community engagement.
- Cleanup activities are most effective during certain times of the year, such as post-monsoon season when debris is washed ashore, increasing the need for organized cleanups and tourist-off season in order to minimize interference with local economies and maximize volunteer availability. Scheduling must account for adverse weather conditions that could hinder outdoor activities and user participation. Peak volunteer participation often occurs during weekends, holidays, and school breaks, which should be considered when scheduling events and platform promotions.
- The project must meet specific milestones to comply with funding agreements and secure continued financial support, budget allocations may need to be utilized within certain fiscal periods, affecting procurement and resource allocation timelines.
- For educators and students to effectively use the platform, align releases of educational resources with the start of academic terms or semesters, have maximum student involvement in the events, and utilize times such as summer breaks for increased involvement in cleanup events and platform testing. Also keeping an eye out for local events that might impact user availability and modifying event schedules to guarantee attendance, arranging worldwide cooperation and virtual gatherings while taking time zone differences into account to serve a variety of user bases.

6g Budget Constraints

Budgetary restrictions that affect the platform's development, deployment, and scope apply to the OceanSweep project. The financial constraints are contingent upon funding sources, resource availability, and cost considerations. As such, meticulous planning and prioritizing are vital to guarantee the triumph of the project.

- The project is mostly dependent on grants, donations, and contributions from non-profit groups and environmental agencies because it is a community-driven environmental initiative, the flexibility in budget allocation is limited by the monetary constraints and possible restrictions on how the funds can be used. Software development costs, such as those for project managers, developers, and designers, must be kept to a minimum, the project shall, as previously stated in section 6a, make every effort to use open-source technology in order to lower licensing fees and software costs. Maintenance, domain registration, server hosting, and other ongoing expenses need to be paid for within the budget that has been set up.
- Limited funding for platform marketing means that low-cost tactics, including social media campaigns, collaborations with environmental groups, and community outreach initiatives, are required. Although cloud services are scalable, using them more frequently can result in higher prices. The project has to choose hosting options that strike a balance between cost and functionality, budgetary reserves should be set aside for unforeseen expenses such as emergency repairs, security flaws, or modifications to third-party services. Supporting users may come with expenses for building extensive self-help resources or manning help desks. Planning for long-term funding sources is necessary for the project to finance updates and upkeep after the original development stage.

7 Naming Conventions and Definitions

7a Definitions of Key Terms

To ensure clarity and a common understanding among all stakeholders involved in the OceanSweep project, this section outlines the key terms, notations, and data structures used throughout the document.

1. Waste Reporting

- **Definition:** The process by which users submit information about sightings of marine debris or pollution through the OceanSweep platform. This includes details such as the type of waste, location, quantity, and any relevant observations.

2. Cleanup Event

- **Definition:** An organized activity coordinated through the OceanSweep platform where volunteers gather to remove waste from oceans, beaches, or

coastal areas. Events include specific dates, times, locations, and details on required equipment or skills.

3. Volunteer

- Definition: An individual who participates in cleanup events or contributes to the OceanSweep community without monetary compensation, motivated by environmental conservation.

4. Diver

- Definition: A user who engages in underwater activities and contributes to OceanSweep by reporting waste sightings beneath the ocean surface and participating in underwater cleanup efforts.

5. Environmental Group

- Definition: A non-profit organization focused on environmental conservation, particularly marine ecosystems. These groups use OceanSweep to organize events, mobilize volunteers, and share educational content.

6. Interactive Map

- Definition: A real-time digital map integrated into the OceanSweep platform that displays reported waste locations, upcoming cleanup events, and areas of interest. It allows users to visualize data geographically.

7. Community Forum

- Definition: A feature within OceanSweep where users engage in discussions, share experiences, ask questions, and collaborate on topics related to marine conservation and pollution reduction.

8. Educational Resources

- Definition: Materials provided on the OceanSweep platform aimed at increasing awareness and understanding of marine pollution issues. This includes articles, videos, tutorials, and interactive modules created by environmental experts.

7b UML and Other Notation Used in This Document

Use Case Diagrams: These diagrams will represent the system's functionality and interactions between external entities (actors) and the system itself. They will use standard UML notations:

- Actors: Represented as stick figures to indicate users or external systems.
- Use Cases: Ovals to denote individual functionalities or interactions.
- System Boundary: A rectangle enclosing the use cases, defining the scope of the system.

7c Data Dictionary for Any Included Models

1. User Profile

Table 3 User Profile

Field Name	Type	Description
UserID	Integer	Unique identifier for each user
Name	String	Full name of the user
Email	String	User's email address
Password	String	Encrypted password for authentication
Roles	Enum	Designation of user's role
Badges	Array of Integers	List of BadgeIDs earned by the user
ProfilePicture	File Path or URL	User's profile image
dateJoined	DateTime	Date when the user registered

2. Waste Report

Table 4 Waste Report

Field Name	Type	Description
ReportID	Integer	Unique Identifier for each waste report

ReporterID	Integer	UserID of the reporting user
LocationLatitude	Decimal	Latitude coordinate of the waste location
LocationLongitude	Decimal	Longitude coordinate of the waste location
WasteType	String	Type of waste reported
Quantity	Integer	Estimated quantity of waste
Description	String	Additional details about the waste
DateTimeReported	DateTime	Timestamp of when the report was submitted
Image	File Path or URL	Photo of the waste

3. Cleanup Report

Table 5 Cleanup Report

Field Name	Type	Description
EventID	Integer	Unique identifier for each event
OrganizerID	Integer	UserID of the event organizer
EventName	String	Title of the event
Description	String	Details about the event
LocationLatitude	Decimal	Latitude coordinate of the event location
LocationLongitude	Decimal	Longitude coordinate of the event location

EventDateTime	Date Time	Scheduled date and time of the event
MaxParticipants	Integers	Maximum number of participants
Participants	Array of Integers	List of UserIDs signed up for the event
EquipmentNeeded	String	Required equipment for the event
Status	Enum	Current status of the event

8 Relevant Facts and Assumptions

8a Facts

Marine pollution is a serious worldwide problem that endangers human health, ecosystems, and marine life. The United Nations Environment Programme (UNEP) estimates that 8 million tons of plastic debris end up in the world's oceans annually. About 80% of this astounding quantity of pollution comes from land-based sources, with the remaining 20% coming from marine activities. Large garbage patches, like the Great Pacific Garbage Patch, which is thought to cover an area twice the size of Texas, have formed as a result of the buildup of waste.

More than 800 marine species are negatively impacted by marine pollution, and plastic waste can lead to entanglement, ingestion, and habitat disruption. Notably, among the most impacted are marine mammals, seabirds, and sea turtles. According to the World Economic Forum, the effects of ocean pollution on enterprises, tourism, and fisheries can cost the world economy up to \$13 billion yearly. The decline in marine health notably affects communities who depend on clean oceans for their livelihoods, such as fishermen and tourism industry workers.

Engaging the community has shown to be a successful tactic in the fight against marine pollution. Local pollution levels have been successfully lowered and public knowledge of the value of marine conservation has increased thanks to volunteer-driven cleanup activities. For example, millions of volunteers worldwide participate in global events

such as International Coastal Cleanup Day, which gather hundreds of tons of rubbish from beaches and rivers.

Technological developments have made these efforts easier by improving data collecting and collaboration. With more than 5 billion individuals using the internet globally—many of them via mobile devices—digital platforms have a significant potential to inspire community action. The cost-effectiveness and collaborative nature of open-source technologies have made them popular, which makes them perfect for creating platforms such as OceanSweep.

There are already ways to report debris and monitor cleanup efforts, like with Marine Debris Tracker and Clean Swell. Nevertheless, these platforms frequently lack all-inclusive capabilities that integrate community interaction, waste reporting, event organization, and educational materials. This disparity emphasizes the requirement for a comprehensive approach that can handle various facets of mitigating marine pollution.

8b Assumptions

It is anticipated that most target users—volunteers, educators, environmental organizations, and members of the community—have access to internet-enabled devices, such as PCs, smartphones, or tablets, and are familiar enough with basic digital literacy to use the OceanSweep platform. It is assumed that users are driven to take part in marine conservation initiatives, ready to report waste sightings, organize cleanup events, and make use of the platform's instructional tools.

Technically speaking, it is considered that the standardized frameworks and open-source technologies required to construct the platform are accessible and appropriate. Throughout the course of the project, it is assumed that the integration of other services—such as mapping APIs, email and notification services, social networking platforms, and authentication systems—is dependable and practical. It is anticipated that the development team possesses the necessary skills to efficiently execute security protocols, guaranteeing data encryption and adherence to pertinent data protection laws such as the CCPA and GDPR.

In terms of resources, it is considered that enough money and personnel will be available to finish the project within the allotted spending limit and timeline. It is anticipated that budgetary restrictions will be manageable with careful planning, a list of the most important features to prioritize, and the adoption of affordable alternatives like open-source software. It is also believed that stakeholders, such as funding agencies and environmental organizations, will continue to support this effort.

By being aware of these presumptions, the OceanSweep project team hopes to foresee possible obstacles and proactively devise solutions. This will help to ensure that the platform is successfully developed, deployed, and adopted in order to reduce beach and ocean pollution through community-driven initiatives.

II Requirements

9 Product Use Cases

9a Use Case Diagrams

1. Reporting Trash Location

Actors: User (Trash Reporter), System

Use Case:

The user reports a trash location by entering the details (location, description, image).

The system stores the report and notifies relevant volunteers and authorities.

2. Volunteer Registration

Actors: Volunteer, System

Use Case:

A volunteer registers for the platform.

The system stores the volunteer's details and availability.

3. Organizing Cleanup Events

Actors: Coordinator, Volunteers, System

Use Case:

The coordinator views reported trash locations.

The system schedules an event and notifies volunteers.

Volunteers confirm participation.

4. Waste Removal Coordination

Actors: Waste Removal Authority, System

Use Case:

The system sends a notification to the waste authority with trash location details.

The authority marks trash as removed once cleared.

5. Notification System

Actors: User (Reporter), Volunteers, Waste Authority, System

Use Case:

The system sends notifications to relevant users based on trash reports, upcoming events, and completion of trash removal.

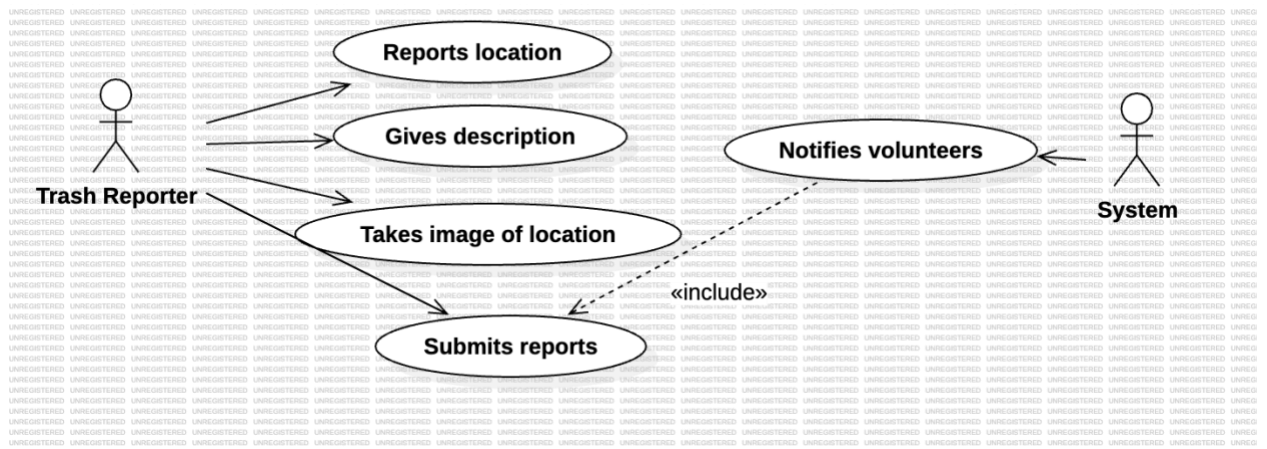


Figure 3 Reporting trash location

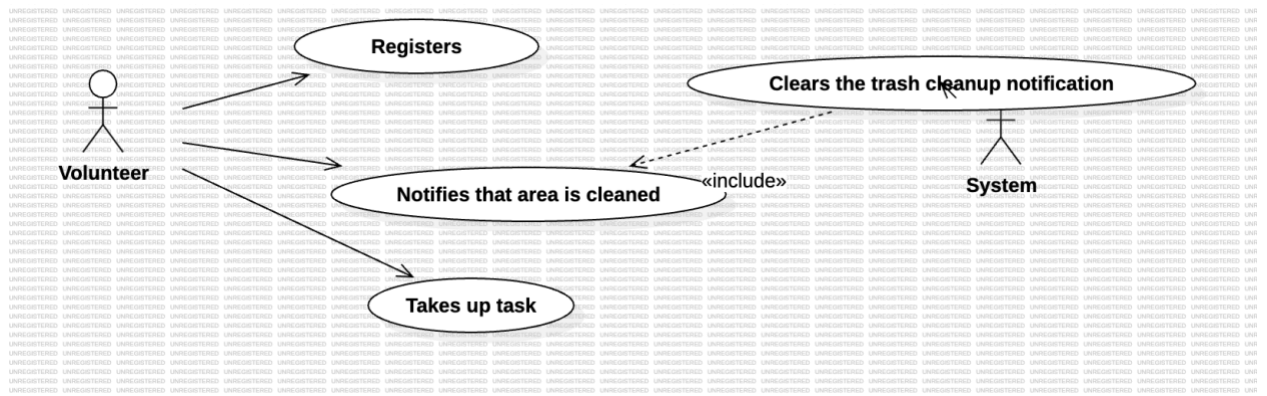


Figure 4 Volunteer Registration

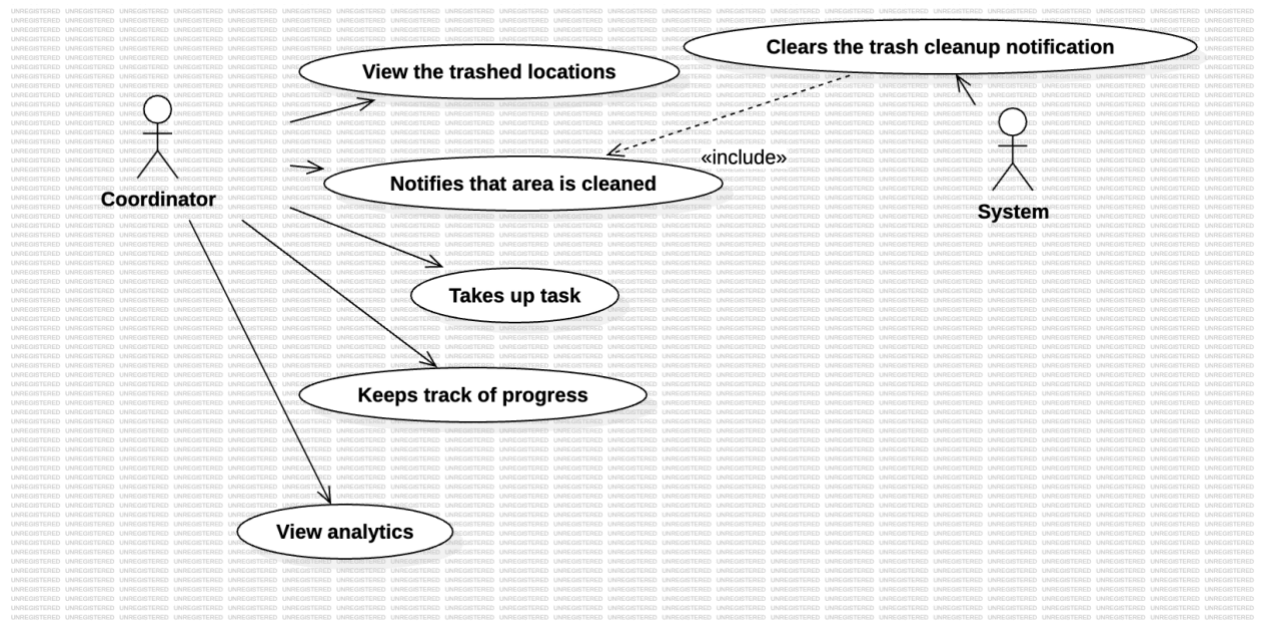


Figure 5 Organizing cleanup events

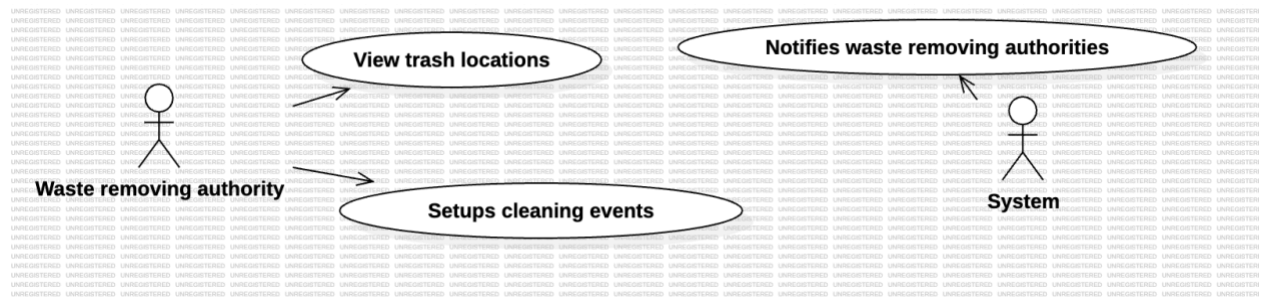


Figure 6 Waste removing coordination

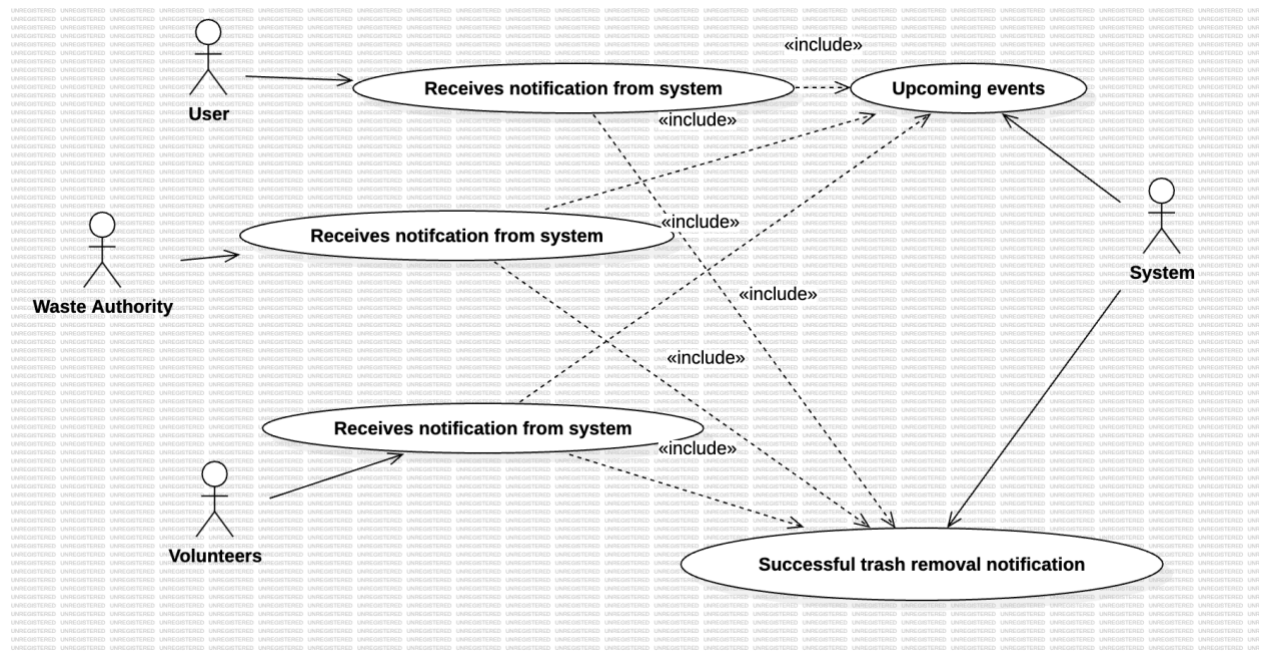


Figure 7 Notification System

9b Product Use Case List

Table 6 Product use case list

Use Case ID	Use Case Name	Description	Actors	Cross-Reference	Notes
UC-01	Reporting Trash Location	User reports a trash location by entering location, description, and image; system notifies relevant volunteers and authorities	User (Trash Reporter), System	System Notifications	Includes image upload and geolocation
UC-02	Volunteer Registration	Volunteer registers on the platform; system stores volunteer details and availability	Volunteer, System	Registration Flow	Requires availability input
UC-03	Organizing Cleanup Events	Coordinator views trash reports, schedules events, and sends volunteer notifications for confirmation of participation	Coordinator, Volunteers, System	Event Scheduling	Event notifications to multiple parties
UC-04	Waste Removal Coordination	System notifies the waste authority of trash locations, which marks locations as cleared after removal	Waste Removal Authority, System	Trash Status Tracking	Notification and status updates
UC-05	Notification System	System sends notifications based on trash reports, event schedules, and trash removal completion	User (Reporter), Volunteers, Waste Authority, System	Notification Flow	Notification customization

9c Individual Product Use Cases

Use Case: Reporting Trash Location

Use Case ID: UC-01 Name: Reporting Trash Location

Pre-Conditions:

- The user must be logged in to report a trash location.
- The system should be online and able to accept reports.

Post-Conditions:

- The trash report is stored in the system.
- Relevant volunteers and authorities are notified.

Initiated By: User (Trash Reporter)

Triggering Event: User initiates the trash report.

Additional Actors: System

Sequence of Events

1. User initiates the report by selecting "Report Trash" on the platform.
2. System displays a form to enter location, description, and image.
3. User fills in the details and submits the report.

4. System saves the report and sends notifications to relevant volunteers and authorities.

Alternatives: If the user cannot provide an image, the system allows submission with only location and description.

Exceptions: If the system is offline, the report cannot be submitted, and the user receives an error message.

Use Case: Volunteer Registration

Use Case ID: UC-02

Name: Volunteer Registration

Pre-Conditions

The system must be online and accepting new volunteers.

Post-Conditions

The volunteer's details are stored, and they are now registered on the platform.

Initiated By:

Volunteer

Triggering Event

Volunteer initiates registration.

Additional Actors: System

Sequence of Events

1. Volunteer selects "Register" on the platform.
2. System displays the registration form requesting personal details and availability.
3. Volunteer fills out the form and submits it.
4. System stores the volunteer's details and confirms registration.

Alternatives

If the volunteer has previously registered, the system prompts them to log in instead.

Exceptions

If required fields are left blank, the system prompts the volunteer to complete the form.

Use Case: Organizing Cleanup Events

Use Case ID: UC-03

Name: Organizing Cleanup Events

Pre-Conditions

- There are reported trash locations to organize events for.
- The coordinator has access to the platform.

Post-Conditions

- A cleanup event is scheduled.
- Volunteers are notified and can confirm their participation.

Initiated By

Coordinator

Triggering Event

Coordinator views trash reports to organize an event.

Additional Actors

System, Volunteers

Sequence of Events

1. Coordinator accesses the trash report list and selects a location to organize an event.

2. System presents an option to schedule the event.
3. Coordinator selects a date, time, and available volunteers, then confirms the event.
4. System sends notifications to selected volunteers for confirmation.
5. Volunteers respond by confirming or declining participation.
6. System updates the event status based on volunteer responses.

Alternatives

- If volunteers decline, the coordinator can reschedule or select different volunteers.

Exceptions

- If no volunteers are available, the system notifies the coordinator and allows them to choose a new date.

10 Functional Requirements

Requirement F-1

ID: F-1

Name: Reporting Trash Location

Description: Users must be able to report trash locations by entering details such as location, description, and uploading an image.

Rationale: Accurate reporting is crucial for mobilizing cleanup efforts and keeping the community informed about pollution.

Fit Criterion: Users should successfully submit a report within 2 minutes, and the report should be stored and notifications sent within 5 seconds.

Acceptance Tests: F-11 (Trash Reporting Test), F-12 (Notification Test)

Requirement F-2

ID: F-2

Name: Volunteer Registration

Description: Volunteers must be able to register on the platform, providing their details and availability.

Rationale: Building a volunteer database is essential for organizing cleanup efforts and ensuring proper communication.

Fit Criterion: Volunteers should be able to register within 3 minutes, and their details must be accessible by the system immediately.

Acceptance Tests: F-21 (Volunteer Registration Test), F-22 (Availability Test)

Requirement F-3

ID: F-3

Name: Organizing Cleanup Events

Description: Coordinators must be able to view reported trash locations and schedule cleanup events, notifying volunteers accordingly.

Rationale: Coordinated cleanup events maximize efficiency and community involvement in pollution removal efforts.

Fit Criterion: Coordinators should schedule an event within 5 minutes and receive confirmations from volunteers within 2 minutes of notification.

Acceptance Tests: F-31 (Event Scheduling Test), F-32 (Volunteer Confirmation Test)

Requirement F-4

ID: F-4

Name: Waste Removal Coordination

Description: The system must notify the waste removal authority of trash location details and allow them to mark trash as removed once cleared.

Rationale: Effective communication with waste authorities is vital for ensuring timely waste removal.

Fit Criterion: Notifications to the waste authority must be sent within 5 seconds of a cleanup event, and the authority should be able to mark trash as removed within 1 minute.

Acceptance Tests: F-41 (Waste Authority Notification Test), F-42 (Removal Confirmation Test)

Requirement F-5

ID: F-5

Name: Tracking Cleanup Progress

Description: Coordinators must track the progress of scheduled cleanups, with the system updating the status of trash reports based on events and volunteer input.

Rationale: Monitoring progress is essential for evaluating cleanup efforts and making data-driven improvements.

Fit Criterion: Status updates should reflect in the system within 2 minutes of volunteer input or event completion.

Acceptance Tests: F-51 (Progress Tracking Test), F-52 (Status Update Test)

Requirement F-6

ID: F-6

Name: Notification System

Description: The system must send notifications to users based on trash reports, upcoming events, and completion of trash removal.

Rationale: Timely notifications enhance user engagement and facilitate effective communication.

Fit Criterion: Notifications must be sent within 10 seconds of relevant events occurring in the system.

Acceptance Tests: F-61 (User Notification Test), F-62 (Event Notification Test)

Requirement F-7

ID: F-7

Name: Analytics Dashboard

Description: The system must provide a dashboard for coordinators displaying analytics on cleanups, volunteer participation, and trash reports.

Rationale: An analytics dashboard helps coordinators assess the effectiveness of cleanup efforts and volunteer engagement.

Fit Criterion: The dashboard should load within 3 seconds and provide real-time analytics on all relevant metrics.

Acceptance Tests: F-71 (Dashboard Load Test), F-72 (Analytics Accuracy Test)

11 Data Requirements

Requirement D-1

ID: D-1

Name: Trash Report Data Structure

Description: The system must define a trash report data structure that includes fields for location (latitude and longitude), description (text), image (file upload), and

timestamp (date and time).

Rationale: A standardized data structure ensures consistent reporting and effective data management for trash sightings.

Fit Criterion: The trash report must be stored in the database within 2 seconds of submission, with all fields validated for completeness.

Acceptance Tests: D-11 (Trash Report Data Structure Test), D-12 (Field Validation Test)

Requirement D-2

ID: D-2

Name: Volunteer Profile Data Structure

Description: The system must include a volunteer profile data structure containing fields for name, contact information (email and phone), availability (days and hours), and areas of expertise (text).

Rationale: A well-defined volunteer profile allows for better matching of volunteers to cleanup events based on their availability and skills.

Fit Criterion: Volunteer profiles must be retrievable and display all information within 3 seconds of a request.

Acceptance Tests: D-21 (Profile Retrieval Test), D-22 (Profile Completeness Test)

Requirement D-3

ID: D-3

Name: Cleanup Event Data Structure

Description: The system must define a cleanup event data structure that includes fields for event name, date and time, location (latitude and longitude), list of volunteers, and status (scheduled, ongoing, completed).

Rationale: Structuring event data enables effective planning and coordination of cleanup efforts, ensuring all relevant information is easily accessible.

Fit Criterion: Event data must be created and accessible within 2 seconds, with all fields validated.

Acceptance Tests: D-31 (Event Creation Test), D-32 (Event Status Update Test)

Requirement D-4

ID: D-4

Name: Waste Removal Authority Data Structure

Description: The system must include a data structure for waste removal authorities that includes fields for authority name, contact information, and response times for reported trash locations.

Rationale: Having structured data on waste authorities enables quick communication

and efficient waste removal processes.

Fit Criterion: Authority data must be updated in real-time, with changes reflected within 5 seconds in the system.

Acceptance Tests: D-41 (Authority Data Update Test), D-42 (Authority Contact Test)

Requirement D-5

ID: D-5

Name: Notification Data Structure

Description: The system must define a notification data structure that includes fields for notification type (e.g., trash report, event reminder), recipient (user ID), timestamp, and message content.

Rationale: A clear structure for notifications ensures timely and relevant communication with users, enhancing engagement.

Fit Criterion: Notifications must be generated and queued for delivery within 1 second of the triggering event.

Acceptance Tests: D-51 (Notification Generation Test), D-52 (Notification Delivery Test)

Requirement D-6

ID: D-6

Name: Educational Resource Data Structure

Description: The system must include a data structure for educational resources that includes fields for resource title, content (text or media), category (e.g., plastic pollution, marine life), and source (author or organization).

Rationale: A structured database of educational resources supports users in understanding the importance of marine conservation and pollution reduction.

Fit Criterion: Educational resources must be retrievable and displayed within 3 seconds of a user query.

Acceptance Tests: D-61 (Resource Retrieval Test), D-62 (Resource Content Test)

12 Performance Requirements

12a Speed and Latency Requirements

Requirement S-1

ID: S-1

Name: Trash Report Submission Speed

Description: The system must process and store a submitted trash report within 2

seconds of the user's submission.

Rationale: Quick processing of trash reports ensures timely notifications to volunteers and authorities, facilitating prompt action on pollution issues.

Fit Criterion: 95% of submitted reports should be processed within the specified time frame.

Acceptance Tests: S-11 (Trash Report Submission Speed Test)

Requirement S-2

ID: S-2

Name: Volunteer Profile Retrieval Speed

Description: The system must retrieve and display volunteer profiles within 3 seconds when requested by coordinators or volunteers.

Rationale: Fast retrieval of volunteer profiles allows coordinators to efficiently match volunteers with cleanup events, improving overall event organization.

Fit Criterion: 90% of profile retrieval requests should meet the speed requirement.

Acceptance Tests: S-21 (Volunteer Profile Retrieval Speed Test)

Requirement S-3

ID: S-3

Name: Event Scheduling Response Time

Description: The system must confirm the scheduling of a cleanup event and notify relevant volunteers within 2 seconds of the coordinator's request.

Rationale: Timely notifications regarding scheduled events enhance volunteer engagement and participation rates.

Fit Criterion: 95% of event scheduling confirmations should occur within the specified time frame.

Acceptance Tests: S-31 (Event Scheduling Response Time Test)

Requirement S-4

ID: S-4

Name: Notification Delivery Speed

Description: The system must deliver notifications to users (trash reporters, volunteers, waste authorities) within 1 second of the triggering event.

Rationale: Immediate notification delivery ensures that users receive timely updates regarding trash reports and upcoming events, fostering active participation.

Fit Criterion: 98% of notifications should be delivered within the required time frame.

Acceptance Tests: S-41 (Notification Delivery Speed Test)

Requirement S-5

ID: S-5

Name: Cleanup Progress Tracking Update Speed

Description: The system must update the status of trash reports and cleanup progress within 2 seconds of receiving input from coordinators or volunteers.

Rationale: Rapid updates on cleanup progress provide users with real-time information, enhancing transparency and engagement in the cleanup efforts.

Fit Criterion: 95% of status updates should occur within the specified time frame.

Acceptance Tests: S-51 (Cleanup Progress Tracking Update Speed Test)

Requirement S-6

ID: S-6

Name: Analytics Dashboard Load Time

Description: The system must load and display the analytics dashboard within 5 seconds for coordinators.

Rationale: Fast loading times for the analytics dashboard allow coordinators to quickly access important data for decision-making and event planning.

Fit Criterion: 90% of dashboard loads should occur within the specified time frame.

Acceptance Tests: S-61 (Analytics Dashboard Load Time Test)

12b Precision or Accuracy Requirements

Requirement A-1

ID: A-1

Name: Trash Location Reporting Accuracy

Description: The system must accurately record and display reported trash locations within a radius of 5 meters of the user's submitted coordinates.

Rationale: High accuracy in reporting trash locations is crucial for effective cleanup operations, ensuring that volunteers and authorities can address pollution precisely where it occurs.

Fit Criterion: 95% of reported locations should be within the specified radius of the actual trash location as verified by follow-up assessments.

Acceptance Tests: A-11 (Trash Location Reporting Accuracy Test)

Requirement A-2

ID: A-2

Name: Waste Volume Measurement Accuracy

Description: The system must provide accurate estimates of the volume of waste reported by users, with a maximum deviation of 10% from actual measurements taken during cleanup events.

Rationale: Accurate waste volume measurements are essential for evaluating the impact of cleanup efforts and for planning future initiatives effectively.

Fit Criterion: 90% of waste volume estimates should fall within the acceptable deviation range during evaluations.

Acceptance Tests: A-21 (Waste Volume Measurement Accuracy Test)

Requirement A-3

ID: A-3

Name: User Feedback Accuracy

Description: The system must accurately reflect user feedback on trash reports and cleanup events, ensuring that the data presented is an accurate representation of the community's input.

Rationale: Accurate feedback is vital for improving the platform and enhancing user satisfaction, as it informs necessary changes and updates to the system.

Fit Criterion: 95% of user feedback should be correctly represented in the system's analytics dashboard.

Acceptance Tests: A-31 (User Feedback Accuracy Test)

Requirement A-4

ID: A-4

Name: Notification Accuracy

Description: The system must ensure that notifications sent to users accurately reflect the events or updates they pertain to, with no more than 5% misclassification or incorrect information.

Rationale: Accurate notifications keep users informed and engaged, reducing confusion and enhancing participation in cleanup efforts.

Fit Criterion: 98% of notifications should accurately match the events they correspond to, as verified through user feedback.

Acceptance Tests: A-41 (Notification Accuracy Test)

Requirement A-5

ID: A-5

Name: Event Scheduling Accuracy

Description: The system must correctly schedule cleanup events based on user

confirmations, with a maximum error rate of 5% in terms of double bookings or incorrect timings.

Rationale: Accurate event scheduling is crucial for effective coordination of cleanup efforts, ensuring that volunteers can participate without conflicts.

Fit Criterion: 95% of scheduled events should accurately reflect user confirmations and timing details.

Acceptance Tests: A-51 (Event Scheduling Accuracy Test)

12c Capacity Requirements

Requirement C-1

ID: C-1

Name: Concurrent User Capacity

Description: The system must support at least 1,000 concurrent users reporting trash locations, coordinating events, and accessing resources without performance degradation.

Rationale: Supporting a high number of concurrent users is essential to ensure that the platform can handle large-scale community engagement, especially during significant events like beach cleanups.

Fit Criterion: The system should maintain a response time of less than 2 seconds for all user interactions when 1,000 users are concurrently active.

Acceptance Tests: C-11 (Concurrent User Load Test)

Requirement C-2

ID: C-2

Name: Trash Reports Handling Capacity

Description: The system must be capable of handling up to 10,000 trash reports submitted within a 24-hour period.

Rationale: High-capacity handling of trash reports is critical to effectively manage user contributions during peak reporting times, such as after major events or in areas with significant pollution.

Fit Criterion: The system should process and store 95% of submitted trash reports within 5 seconds, even under peak load conditions.

Acceptance Tests: C-21 (Trash Reports Processing Test)

Requirement C-3

ID: C-3

Name: Cleanup Event Capacity

Description: The system must support the scheduling of up to 100 simultaneous cleanup events across different geographic locations.

Rationale: Supporting multiple cleanup events simultaneously allows the platform to facilitate broad community engagement and maximize participation in environmental efforts.

Fit Criterion: All scheduled events should be accurately represented on the platform's calendar and accessible to users without delays.

Acceptance Tests: C-31 (Simultaneous Event Scheduling Test)

Requirement C-4

ID: C-4

Name: Data Storage Capacity

Description: The system must provide at least 1 TB of data storage capacity for user-generated content, including trash reports, event details, and educational resources.

Rationale: As the platform grows, it will accumulate significant amounts of data, necessitating ample storage to maintain operational integrity and historical records.

Fit Criterion: The system should support data growth of at least 10% per month without performance degradation or data loss.

Acceptance Tests: C-41 (Data Storage Capacity Test)

Requirement C-5

ID: C-5

Name: User Engagement Capacity

Description: The system must be able to handle up to 50,000 registered users, with features for community discussions, event participation, and resource sharing.

Rationale: A large user base is essential for fostering community-driven efforts and ensuring a wide reach in mobilizing resources for pollution cleanup.

Fit Criterion: The system should support user interactions, such as comments and feedback, with no more than a 1-second latency for 95% of interactions.

Acceptance Tests: C-51 (User Engagement Load Test)

13 Dependability Requirements

13a Reliability Requirements

Requirement R-1

ID: R-1

Name: System Uptime

Description: The system must maintain an uptime of 99.9% to ensure continuous accessibility for users, including real-time reporting, notifications, and event coordination.

Rationale: High availability is critical for OceanSweep's functionality, particularly during peak hours when users rely on the platform for timely reporting and cleanup coordination.

Fit Criterion: The system should be accessible and functional 99.9% of the time, allowing for only 8.76 hours of downtime annually.

Acceptance Tests: R-11 (Scheduled Uptime Monitoring Test), R-12 (Downtime Logging and Analysis Test)

Requirement R-2

ID: R-2

Name: Data Integrity

Description: The system must ensure that all user-submitted data (e.g., trash reports, volunteer registrations, event details) is stored accurately and is retrievable without corruption.

Rationale: Data integrity is essential to provide accurate records and reliable information to users, especially when organizing cleanups or generating analytics for environmental impact.

Fit Criterion: The system should successfully retrieve 100% of stored data in its original form during random data integrity checks.

Acceptance Tests: R-21 (Data Integrity Verification Test), R-22 (Data Backup and Recovery Test)

Requirement R-3

ID: R-3

Name: Error-Free Operation

Description: The system must handle 95% of user operations without errors, such as failed data submissions or processing issues.

Rationale: A high level of operational reliability reduces user frustration and enhances trust in the platform, encouraging consistent engagement from users and volunteers.

Fit Criterion: The system should complete 95% of user operations, including trash reporting, event organization, and notifications, without errors in a 24-hour period.

Acceptance Tests: R-31 (Operational Reliability Test), R-32 (Error Handling Test)

Requirement R-4

ID: R-4

Name: Graceful Failure Handling

Description: In case of unexpected system failures, the platform must provide users with clear feedback messages and log errors for troubleshooting.

Rationale: Ensuring that users receive informative messages during failures can minimize frustration and help guide their actions while technical support addresses the issue.

Fit Criterion: The system should provide clear error messages for 100% of failures and automatically log errors with timestamps and relevant diagnostic information for further analysis.

Acceptance Tests: R-41 (Failure Notification Test), R-42 (Error Logging Verification Test)

13b Availability Requirements

Requirement A-1

ID: AR-11

Name: High Availability

Description: The system must be available 99.9% of the time, allowing for minimal downtime, including planned maintenance windows.

Rationale: High availability is crucial for ensuring that users can rely on the system for timely access to reporting and event coordination. With a high percentage of uptime, OceanSweep will be dependable for volunteers and coordinators.

Fit Criterion: The system should maintain an uptime of 99.9%, allowing no more than 8.76 hours of downtime per year.

Acceptance Tests: AR-11 (Uptime Monitoring Test), AR-12 (Maintenance Downtime Logging Test)

Requirement A-2

ID: AR-2

Name: Rapid Recovery

Description: The system must recover from any unscheduled downtime within 30 minutes, including the restoration of all data and services.

Rationale: Rapid recovery after an unexpected outage ensures minimal disruption to users, maintaining user trust and continuous engagement.

Fit Criterion: In case of unexpected downtime, all services should be restored within 30 minutes, with automated failover and recovery mechanisms ensuring continuity.

Acceptance Tests: AR-21 (Disaster Recovery Test)

13c Robustness or Fault-Tolerance Requirements

Requirement R-1

ID: RF-1

Name: Offline Mode for Reporting

Description: The system should allow users to report trash locations in offline mode, storing the information locally and uploading it once network connectivity is restored.

Rationale: Offline functionality enables users in remote or low-connectivity areas to continue reporting waste without interruptions, improving data collection reliability.

Fit Criterion: When offline, users can save trash reports locally, which will automatically sync with the server upon reconnecting to the internet.

Acceptance Tests: RF-1 (Offline Report Storage Test)

13d Safety-Critical Requirements

Requirement S-1

ID: SC-1

Name: Hazard Notification for Hazardous Waste

Description: The system should automatically flag and notify users when a reported trash location includes hazardous waste (e.g., sharp objects, toxic substances) and provide safety guidelines.

Rationale: Ensuring volunteers are informed of hazardous waste is essential to protect their health and safety during cleanup events. Providing safety guidelines reduces the risk of injury.

Fit Criterion: The system should flag reports containing keywords associated with hazardous materials and automatically send a safety notification to all volunteers registered for the cleanup.

Acceptance Tests: SC-11 (Hazardous Material Detection Test), SC-12 (Automatic Safety Notification Test)

Requirement S-2

ID: SC-2

Name: OSHA Compliance for Safety Procedures

Description: The platform's event organization process should comply with OSHA standards for handling waste, including providing recommended protective gear and disposal methods.

Rationale: Adherence to OSHA standards protects users from health risks associated with waste handling, reducing liability and promoting a safe environment for

volunteers.

Fit Criterion: All cleanup event notifications should include OSHA-compliant safety recommendations, such as wearing gloves and masks, and proper disposal practices.

Acceptance Tests: SC-21 (OSHA Compliance Verification Test), SC-22 (Safety Recommendation Display Test)

Requirement S-3

ID: SC-3

Name: Emergency Assistance Feature

Description: The system should offer a one-click emergency contact option for users to alert authorities if they encounter unsafe conditions during cleanup events.

Rationale: Providing a quick way to contact emergency services ensures that users can seek help immediately if they encounter dangerous situations, enhancing overall safety during events.

Fit Criterion: The system should allow users to trigger an emergency alert, which sends location details to the nearest emergency service with one click.

Acceptance Tests: SC-31 (Emergency Contact Activation Test)

Requirement S-4

ID: S-4

Name: Risk Assessment for Large Cleanups

Description: For large cleanup events, the system should perform a risk assessment, factoring in location type and potential hazards, and advise coordinators on the necessary safety protocols.

Rationale: Assessing risk based on event scale and location enhances volunteer safety by tailoring protocols to specific event conditions.

Fit Criterion: The system should automatically generate a safety protocol checklist based on the size and location of the cleanup, ensuring that all safety measures are communicated to participants.

Acceptance Tests: SC-41 (Risk Assessment Generation Test)

14 Maintainability and Supportability Requirements

14a Maintenance Requirements

Requirement M-1

ID: MS-1

Name: Regular System Updates and Patch Management

Description: The system must support regular updates, including bug fixes, security

patches, and feature improvements, without significant downtime. Maintenance schedules should be configurable to avoid peak usage hours.

Rationale: Regular updates ensure the system remains secure, functional, and up-to-date, enhancing user experience and maintaining compliance with security standards. Configurable schedules minimize disruption for users.

Fit Criterion: The system allows maintenance windows to be scheduled outside peak usage times and supports automated updates that do not exceed a downtime of 10 minutes.

Acceptance Tests: MS-11 (Automated Update Test)

Requirement M-2

ID: MS-2

Name: Remote Maintenance Capability

Description: The system should support remote maintenance, allowing authorized personnel to access and perform necessary maintenance tasks off-site.

Rationale: Remote maintenance ensures quick response to issues without the need for on-site presence, reducing maintenance time and enabling immediate support when needed.

Fit Criterion: Authorized personnel can securely access the system remotely to perform diagnostics, updates, and troubleshooting.

Acceptance Tests: MS-21 (Remote Access Verification Test), MS-22 (Remote Maintenance Test)

14b Supportability Requirements

Not Applicable

14c Adaptability Requirements

Requirement A-1

ID: ADR-1

Name: Cross-Browser Compatibility

Description: The OceanSweep platform should be accessible and fully functional across popular web browsers, including Chrome, Firefox, Safari, and Edge, to ensure a consistent user experience.

Rationale: Users may access the platform from various browsers; ensuring cross-browser compatibility will improve usability and accessibility across different systems.

Fit Criterion: The platform should successfully pass usability and functionality tests on the latest stable versions of Chrome, Firefox, Safari, and Edge, with all core features operating consistently.

Acceptance Tests: ADR-1: Compatibility Test for cross-browser

14d Scalability or Extensibility Requirements

Requirement S-1

ID: SE-1

Name: Data Storage Scalability

Description: OceanSweep must support scalable data storage solutions to accommodate an increasing number of trash reports, user registrations, and event records as usage grows.

Rationale: As more users and reports are added to the platform, a scalable data solution will ensure system reliability and maintain performance without excessive downtime or data loss.

Fit Criterion: The platform should maintain optimal performance with a database that scales to handle up to 1 million trash reports and 500,000 user records without latency exceeding 1 second on average.

Acceptance Tests: SE-11: Data Load Test for Trash Reports

Requirement S-2

ID: SE-2

Name: Volunteer Network Extensibility

Description: The system should be designed to support an extensible volunteer management module to handle a larger volunteer network and additional roles as the program expands.

Rationale: As the platform grows, it may require support for new volunteer roles and expanded geographic areas, ensuring effective coordination and engagement.

Fit Criterion: The volunteer management module should support up to 50,000 volunteers across various roles and geographical regions, with role-based access control for system management.

Acceptance Tests: SE-21: Role-Based Access Control Test

14e Longevity Requirements

Requirement L-1

ID: L-1

Name: System Lifetime Expectancy

Description: OceanSweep should be designed to operate effectively for a minimum of 10 years with regular maintenance, updates, and scalability accommodations.

Rationale: The platform aims to be a long-term solution for ocean waste management and community engagement. Designing for longevity ensures sustainable operation without requiring a complete overhaul or redevelopment within

the expected timeframe.

Fit Criterion: The system architecture should allow for compatibility with updates in technology standards and expansions in user capacity over the next decade, maintaining at least 95% operational efficiency.

Acceptance Tests: L-11: System Durability Test

15 Security Requirements

15a Access Requirements

Requirement A-1

ID: ARQ-1

Name: Role-Based Access Control

Description: OceanSweep must implement role-based access control to ensure that users, volunteers, coordinators, and authorities only access data and functionality pertinent to their roles.

Rationale: Limiting access according to user roles is essential to protect sensitive information and prevent unauthorized actions.

Fit Criterion: The system should provide different access levels, restricting user access to only the functions and data permitted for their role, verified by successful access control tests.

Acceptance Tests: ARQ-11: Role Access Verification Test

15b Integrity Requirements

Requirement I-1

ID: I-1

Name: Data Integrity Checks

Description: The system must conduct regular integrity checks on all stored data, ensuring no unintended changes occur to user information, trash reports, and other critical data.

Rationale: Data accuracy is essential for operational reliability and trust among users and collaborators.

Fit Criterion: Data integrity checks should run daily, with reports showing less than 0.01% data corruption across all records in a year.

Acceptance Tests: I-11: Scheduled Integrity Check Test; I-12: Data Corruption Handling Test.

15c Privacy Requirements

Not Applicable

15d Audit Requirements

Not Applicable

15e Immunity Requirements

Requirement IM-1

ID: IM-1

Name: Malware Protection Measures

Description: The system should include protection against malware, including routine scans and a firewall to prevent viruses, worms, and Trojan horses from compromising system security.

Rationale: Protecting against malware ensures system stability, data security, and user safety.

Fit Criterion: OceanSweep must pass security assessments with malware detection rates above 99%, with continuous protection against known threats.

Acceptance Tests: IM-11: System Malware Vulnerability Scan Test

16 Usability and Humanity Requirements

16a Ease of Use Requirements

ID#: EU-1

Name: User-Friendly Interface

Description: Interface allows easy access to main features within 2-3 clicks.

Rationale: Simplifies navigation for all users.

Fit Criterion: 90% of new users navigate main features in under 3 clicks.

Acceptance Tests: EU-11: Verify users access primary features within 3 clicks.

ID#: EU-2

Name: Quick Setup

Description: Users complete setup in under 5 minutes.

Rationale: Reduces entry barriers and supports ease of use.

Fit Criterion: 95% of users complete setup without help in under 5 minutes.

Acceptance Tests: EU-21: Test setup time for average users.

16b Personalization and Internationalization Requirements

ID#: PI-1

Name: Language Selection

Description: Users can select preferred language from a list of supported languages.

Rationale: Improves accessibility and user experience for a global audience.

Fit Criterion: Users can switch languages from settings, and 95% of text appears in the selected language.

Acceptance Tests: PI-11: Verify language selection changes all interface text.

16c Learning Requirements

ID#: LR-1

Name: User Onboarding Tutorial

Description: A step-by-step tutorial guides new users through basic features on their first login.

Rationale: Helps users quickly learn core functions, reducing initial confusion.

Fit Criterion: At least 90% of new users complete the tutorial within the first session.

Acceptance Tests:

- **LR-11:** Confirm tutorial activates on first login and covers basic functions.

16d Understandability and Politeness Requirements

ID#: UP-1

Name: Clear Messaging and Instructions

Description: All system messages and instructions use plain, concise language to ensure user comprehension.

Rationale: Reduces confusion and errors, leading to a smoother user experience.

Fit Criterion: 90% of users should report understanding messages and instructions without external help.

Acceptance Tests: UP-11: Verify system messages are clear and concise.

16e Accessibility Requirements

ID#: AR-1

Name: Screen Reader Compatibility

Description: Ensure all system interfaces are fully compatible with screen readers to support visually impaired users.

Rationale: Allows visually impaired users to effectively navigate and use the system.

Fit Criterion: System passes compatibility testing with major screen readers (e.g., NVDA, JAWS) for 100% of essential navigation and interaction elements.

Acceptance Tests: AREQ-11: Test screen reader functionality across all key pages and actions.

16f User Documentation Requirements

Not Applicable

16g Training Requirements

Not Applicable.

17 Look and Feel Requirements

17a Appearance Requirements

ID#: ARP-1

Name: Brand Color Scheme Compliance

Description: The system must use the designated brand colors across all interfaces.

Rationale: Consistency with corporate branding reinforces brand identity and enhances user recognition.

Fit Criterion: All visual elements comply with the designated brand colors as specified in the corporate style guide.

Acceptance Tests:

- **ARP-11:** Confirm colors match the official brand palette on main screens.
- **ARP-12:** Validate consistency across buttons, links, and backgrounds.

17b Style Requirements

ID#: STY-1

Name: Professional User Interface

Description: The system interface should convey a professional and clean aesthetic suitable for corporate users.

Rationale: A professional style enhances user trust and confidence in the system's reliability.

Fit Criterion: The interface has minimalistic, modern design elements with neutral color schemes and organized layouts that prioritize readability.

Acceptance Tests:

STY-11: Confirm the use of neutral tones and simple design elements on primary screens.

ID#: STY-2

Name: Friendly Interaction Cues

Description: Interactive messages and cues should be friendly and approachable to encourage ease of use.

Rationale: Friendly interactions make the system more approachable, improving the user experience, especially for first-time users.

Fit Criterion: System messages and prompts are written in an encouraging, conversational tone.

Acceptance Tests:

STY-21: Verify that error messages use friendly language and suggest solutions.

18 Operational and Environmental Requirements

18a Expected Physical Environment

Not Applicable

18b Requirements for Interfacing with Adjacent Systems

Not Applicable

18c Productization Requirements

ID# - PR-1

Name: Mobile and Desktop Compatibility

Description: The system must be compatible with Android and iOS platforms, along with desktop platforms (Windows, macOS, Linux).

Rationale: Ensuring multi-platform compatibility increases accessibility for a broader user base.

Fit Criterion: The product functions seamlessly on Android and iOS devices, as well as Windows, macOS, and Linux environments without compatibility issues.

Acceptance Tests: ADR-1: Compatibility Test for cross-browser

18d Release Requirements

ID# - RR-1

Name: Product Release Cycle

Description: The system shall follow a bi-annual release cycle, with each release including a full update package compatible with all supported platforms (Android, iOS, Windows, macOS, and Linux).

Rationale: Regular updates ensure that the product remains secure, compatible with platform changes, and incorporates user feedback and new features.

Fit Criterion: The system is updated and released bi-annually, with documented release notes and platform-specific installation instructions.

Acceptance Tests: RR-11: Review of release schedule adherence

19 Cultural and Political Requirements

19a Cultural Requirements

Not Applicable

19b Political Requirements

Not Applicable

20 Legal Requirements

20a Compliance Requirements

ID# - CR-1

Name: Legal Compliance

Description: The system shall comply with relevant data protection and privacy laws.

Rationale: Ensures user data security and legal alignment.

Fit Criterion: Passes compliance audits and legal reviews.

Acceptance Tests: CR-11 Documented audit reports and legal approvals.

20b Standards Requirements

ID# - SR-1

Name: Standards Compliance

Description: The system shall adhere to relevant industry standards for software quality and data handling.

Rationale: Ensures consistency, quality, and interoperability across similar platforms.

Fit Criterion: Meets or exceeds standards for data handling and software quality as per industry norms.

Acceptance Tests:

SR-11. Verification report against industry standards.

20c Requirements – Test Correspondence Summary

Table 7 - Requirements - Acceptance Tests Correspondence

Test ID	Test Name	Requirements
F-11	Trash reporting test	1
F-12	Notification test	2
F-21	Volunteer Registration Test	3
F-31	Event Scheduling test	4
F-32	Volunteer confirmation test	5
F-41	Waste Authority Notification test	6
F-42	Removal Confirmation test	7
F-51	Progress tracking test	8
F-52	Status update test	9
F-61	User Notification test	10
F-62	Event notification test	11
F-71	Dashboard load test	12
F-72	Analytics Accuracy test	13
D-11	Trash report data structure test	14
D-12	Field Validation test	15
D-21	Profile retrieval test	16
D-22	Profile Completeness test	17
D-31	Event Creation test	18
D-32	Event Status Update test	19
D-41	Authority Data Update test	20
D-42	Authority Contact Test	21
D-51	Notification Generation Test	22
D-52	Notification Delivery Test	23
D-61	Resource retrieval test	24
D-62	Resource content test	25
S-11	Trash report submission speed test	26
S-21	Volunteer Profile Retrieval Speed Test	27
S-31	Event scheduling resp t.t	28
S-41	Notification Delivery Speed Test	29

S-51	Cleanup Progress Tracking update speed test	30
S-61	Analytics Dashboard Load Time Test	31
A-11	Trash location reporting accuracy test	32
A-21	Waste Volume Measurement Accuracy Test	33
A-31	User Feedback Accuracy test	34
A-41	Notification Accuracy test	35
A-51	Event Scheduling Accuracy test	36
C-11	Concurrent User Load test	37
C-21	Trash Reports Processing test	38
C-31	Simultaneous Event Scheduling test	39
C-41	Data Storage Capacity test	40
C-51	User Engagement Load test	41
R-11	Scheduled Uptime Monitoring Test	42
R-12	Downtime logging and analysis test	43
R-21	Data Integrity Verification Test	44
R-22	Data Backup and Recovery test	45
R-31	Operational Reliability test	46
R-32	Error Handling test	47
R-41	Failure Notification test	48
R-42	Error Logging Verification test	49

AR-11	Uptime Monitoring test	50
AR-12	Maintenance Downtime logging test	51
AR-21	Disaster Recovery test	52
RF-1	Offline Report Storage test	53
SC-11	Hazardous material detection test	54
SC-12	Automatic Safety Notification test	55
SC-21	OSHA Compliance verification test	56
SC-22	Safety Recommendation Display test	57
SC-31	Emergency Contact Activation test	58
SC-41	Risk Assessment Generation test	59
MS-11	Automated Update test	60
MS-21	Remote Access Verification test	61
MS-22	Remote Maintenance test	62
ADR-1	Compatibility test for cross-browser	63,81
SE-11	Data Load Test for Trash Reports	64
SE-21	Role-Based Access Control Test	65
L-11	System Durability test	66
ARQ-11	Role Access Verification test	67
I-11	Scheduled Integrity Check Test	68
I-12	Data Corruption handling test	69
IM-11	System Malware Vulnerability Scan Test	70
EU-11	Verify within 3 clicks	71
EU-21	Test setup time	72

PI-11	Verify language selection	73
LR-11	Confirm tutorial activation	74
UP-11	Verify system messages are clear	75
AREQ-11	Test screen reader functionality	76
ARP-11	Confirm color match official brand palatte	77
ARP-12	Validate consistency across buttons	78
STY-11	Confirm use of neutral tone	79
STY-21	Verify error message use friendly tone	80
RR-11	Review of release schedule adherence	82
CR-11	Documented audit reports and legal approvals	83
SR-11	Verification report against industry standards	84

Test ID	Test Name	Requirements
F-11	Trash reporting test	1
F-12	Notification test	2
F-21	Volunteer Registration Test	3
F-31	Event Scheduling test	4
F-32	Volunteer confirmation test	5
F-41	Waste Authority Notification test	6
F-42	Removal Confirmation test	7
F-51	Progress tracking test	8
F-52	Status update test	9
F-61	User Notification test	10
F-62	Event notification test	11
F-71	Dashboard load test	12
F-72	Analytics Accuracy test	13
D-11	Trash report data structure test	14
D-12	Field Validation test	15
D-21	Profile retrieval test	16

D-22	Profile Completeness test	17
D-31	Event Creation test	18
D-32	Event Status Update test	19
D-41	Authority Data Update test	20
D-42	Authority Contact Test	21
D-51	Notification Generation Test	22
D-52	Notification Delivery Test	23
D-61	Resource retrieval test	24
D-62	Resource content test	25
S-11	Trash report submission speed test	26
S-21	Volunteer Profile Retrieval Speed Test	27
S-31	Event scheduling resp t.t	28
S-41	Notification Delivery Speed Test	29
S-51	Cleanup Progress Tracking update speed test	30

S-61	Analytics Dashboard Load Time Test	31
A-11	Trash location reporting accuracy test	32
A-21	Waste Volume Measurement Accuracy Test	33
A-31	User Feedback Accuracy test	34
A-41	Notification Accuracy test	35
A-51	Event Scheduling Accuracy test	36
C-11	Concurrent User Load test	37
C-21	Trash Reports Processing test	38
C-31	Simultaneous Event Scheduling test	39
C-41	Data Storage Capacity test	40
C-51	User Engagement Load test	41
R-11	Scheduled Uptime Monitoring Test	42
R-12	Downtime logging and analysis test	43
R-21	Data Integrity Verification Test	44

R-22	Data Backup and Recovery test	45
R-31	Operational Reliability test	46
R-32	Error Handling test	47
R-41	Failure Notification test	48
R-42	Error Logging Verification test	49
AR-11	Uptime Monitoring test	50
AR-12	Maintenance Downtime logging test	51
AR-21	Disaster Recovery test	52
RF-1	Offline Report Storage test	53
SC-11	Hazardous material detection test	54
SC-12	Automatic Safety Notification test	55
SC-21	OSHA Compliance verification test	56
SC-22	Safety Recommendation Display test	57
SC-31	Emergency Contact Activation test	58

SC-41	Risk Assessment Generation test	59
MS-11	Automated Update test	60
MS-21	Remote Access Verification test	61
MS-22	Remote Maintenance test	62
ADR-1	Compatibility test for cross-browser	63,81
SE-11	Data Load Test for Trash Reports	64
SE-21	Role-Based Access Control Test	65
L-11	System Durability test	66
ARQ-11	Role Access Verification test	67
I-11	Scheduled Integrity Check Test	68
I-12	Data Corruption handling test	69
IM-11	System Malware Vulnerability Scan Test	70
EU-11	Verify within 3 clicks	71
EU-21	Test setup time	72
PI-11	Verify language selection	73

LR-11	Confirm tutorial activation	74
UP-11	Verify system messages are clear	75
AREQ-11	Test screen reader functionality	76
ARP-11	Confirm color match official brand palette	77
ARP-12	Validate consistency across buttons	78
STY-11	Confirm use of neutral tone	79
STY-21	Verify error message use friendly tone	80
RR-11	Review of release schedule adherence	82
CR-11	Documented audit reports and legal approvals	83
SR-11	Verification report against industry standards	84

20d Acceptance Test Descriptions

F-11 - Trash Reporting Test

Description: Ensures that the system allows users to report trash locations with precise details, facilitating efficient cleanup operations.

F-12 - Notification Test

Description: Verifies that the notification system works as expected, delivering timely and relevant updates to users.

F-21 - Volunteer Registration Test

Description: Confirms that volunteers can successfully register with accurate profile information for event participation.

F-31 - Event Scheduling Test

Description: Tests the scheduling feature, ensuring that cleanup events can be planned and displayed with correct timings and locations.

F-32 - Volunteer Confirmation Test

Description: Ensures volunteers receive appropriate confirmations after registering for events, including necessary details for participation.

F-41 - Waste Authority Notification Test

Description: Validates that relevant waste management authorities receive notifications about reported trash, ensuring prompt action.

F-42 - Removal Confirmation Test

Description: Confirms that users receive notifications once reported trash has been successfully removed, providing closure.

F-51 - Progress Tracking Test

Description: Checks that progress on trash removal efforts is accurately tracked and visible to stakeholders.

F-52 - Status Update Test

Description: Verifies that status updates related to cleanup operations are timely and accurately reflected within the system.

F-61 - User Notification Test

Description: Ensures users are consistently notified of important updates or actions related to their reports or volunteer activities.

F-62 - Event Notification Test

Description: Confirms that notifications are sent to users regarding upcoming events or volunteer opportunities.

F-71 - Dashboard Load Test

Description: Evaluates the dashboard's load performance to ensure it can handle user traffic without delays.

F-72 - Analytics Accuracy Test

Description: Verifies the accuracy of data presented in the analytics dashboard, confirming that it reflects true metrics.

D-11 - Trash Report Data Structure Test

Description: Ensures that data related to trash reports is structured correctly, making it accessible and organized.

D-12 - Field Validation Test

Description: Checks for validation in data fields, ensuring entries are complete and error-free before submission.

D-21 - Profile Retrieval Test

Description: Tests that users can retrieve their profile information without issues, displaying all registered details correctly.

D-22 - Profile Completeness Test

Description: Ensures that user profiles are complete with required fields, providing a thorough representation of each volunteer's information.

D-31 - Event Creation Test

Description: Confirms that events can be created within the system, complete with scheduling and resource allocation.

D-32 - Event Status Update Test

Description: Verifies that event statuses (e.g., scheduled, completed) are updated correctly in real-time.

D-41 - Authority Data Update Test

Description: Checks that authority-related data is kept up to date, ensuring the correct agencies are informed of cleanup needs.

D-42 - Authority Contact Test

Description: Ensures that relevant authorities can be contacted as needed via the system, facilitating quick responses.

D-51 - Notification Generation Test

Description: Verifies that notifications are generated for significant events, changes, or alerts within the system.

D-52 - Notification Delivery Test

Description: Confirms that notifications are delivered reliably to users, keeping them informed of relevant updates.

D-61 - Resource Retrieval Test

Description: Ensures that users can retrieve resources or information related to cleanup efforts.

D-62 - Resource Content Test

Description: Checks that content provided in resources is accurate, relevant, and accessible.

S-11 - Trash Report Submission Speed Test

Description: Measures the speed of submitting trash reports, ensuring quick processing for efficient cleanup planning.

S-21 - Volunteer Profile Retrieval Speed Test

Description: Tests the retrieval speed of volunteer profiles, ensuring quick access to volunteer information.

S-31 - Event Scheduling Response Time Test

Description: Verifies the responsiveness of event scheduling, ensuring minimal lag in creating and updating events.

S-41 - Notification Delivery Speed Test

Description: Confirms that notifications are delivered promptly to users without delay.

S-51 - Cleanup Progress Tracking Update Speed Test

Description: Measures the update speed of progress tracking, ensuring real-time visibility into cleanup efforts.

S-61 - Analytics Dashboard Load Time Test

Description: Tests the load time of the analytics dashboard to ensure it operates efficiently with minimal delay.

A-11 - Trash Location Reporting Accuracy Test

Description: Verifies that trash locations reported by users are accurate, facilitating effective cleanup planning.

A-21 - Waste Volume Measurement Accuracy Test

Description: Ensures that waste volume estimates are accurate for planning resources and manpower needed.

A-31 - User Feedback Accuracy Test

Description: Confirms that user feedback related to the cleanup process is captured and represented accurately.

A-41 - Notification Accuracy Test

Description: Ensures that notifications sent to users are accurate and relevant to their actions or events.

A-51 - Event Scheduling Accuracy Test

Description: Verifies that event schedules are accurately reflected, ensuring volunteers are informed of the correct details.

C-11 - Concurrent User Load Test

Description: Tests the system's capacity to handle multiple users concurrently without performance degradation.

C-21 - Trash Reports Processing Test

Description: Verifies that the system can process multiple trash reports simultaneously without delays.

C-31 - Simultaneous Event Scheduling Test

Description: Confirms that multiple events can be scheduled at the same time without system conflicts.

C-41 - Data Storage Capacity Test

Description: Ensures that the system's data storage can handle a high volume of records, maintaining performance.

C-51 - User Engagement Load Test

Description: Evaluates the system's ability to handle high user engagement, ensuring stability and speed.

R-11 - Scheduled Uptime Monitoring Test

Description: Monitors system uptime to ensure reliability and availability to users.

R-12 - Downtime Logging and Analysis Test

Description: Confirms that system downtimes are logged accurately for analysis and troubleshooting.

R-21 - Data Integrity Verification Test

Description: Ensures that data within the system is consistent and uncorrupted, providing accurate information.

R-22 - Data Backup and Recovery Test

Description: Verifies that data backups are created and that recovery functions correctly in case of data loss.

R-31 - Operational Reliability Test

Description: Tests the system's reliability under standard operating conditions.

R-32 - Error Handling Test

Description: Ensures that errors within the system are handled gracefully, with user-friendly messages or resolutions.

R-41 - Failure Notification Test

Description: Confirms that users are notified of any critical system failures impacting their operations.

R-42 - Error Logging Verification Test

Description: Verifies that system errors are logged for effective troubleshooting.

AR-11 - Uptime Monitoring Test

Description: Confirms that the system uptime is constantly monitored, maintaining reliable service.

AR-12 - Maintenance Downtime Logging Test

Description: Ensures that downtime due to maintenance is logged and communicated as necessary.

AR-21 - Disaster Recovery Test

Description: Verifies that the system can recover from critical failures or data loss effectively.

RF-1 - Offline Report Storage Test

Description: Confirms that reports can be stored and accessed offline for remote or low-connectivity scenarios.

SC-11 - Hazardous Material Detection Test

Description: Tests the detection of hazardous materials in reported trash, ensuring safe handling.

SC-12 - Automatic Safety Notification Test

Description Verifies that users receive safety notifications automatically for hazardous materials.

SC-21 - OSHA Compliance Verification Test

Description: Ensures compliance with OSHA standards in reporting, handling, and notifications.

SC-22 - Safety Recommendation Display Test

Description: Confirms that safety recommendations are accurately displayed to users.

SC-31 - Emergency Contact Activation Test

Description: Verifies that emergency contacts are notified effectively during critical incidents.

SC-41 - Risk Assessment Generation Test

Description: Ensures that risk assessments are generated accurately based on reported conditions.

MS-11 - Automated Update Test

Description: Verifies that system updates are automated and deploy without user disruption.

MS-21 - Remote Access Verification Test

Description: Confirms that remote access is functional and secure.

MS-22 - Remote Maintenance Test

Description: Ensures maintenance can be conducted remotely without impacting system availability.

ADR-1 - Compatibility Test for Cross-Browser

Description: Ensures system compatibility across multiple browsers, ensuring consistency in user experience.

SE-11 - Data Load Test for Trash Reports

Description: Ensures that the system can handle and load a high volume of trash report data efficiently without performance degradation.

SE-21 - Role-Based Access Control Test

Description: Verifies that role-based access control is implemented correctly, restricting and granting access based on user roles.

L-11 - System Durability Test

Description: Tests the system's durability and resilience to ensure it can withstand prolonged usage and potential stress conditions.

ARQ-11 - Role Access Verification Test

Description: Confirms that assigned roles have appropriate permissions and access levels as per system requirements.

I-11 - Scheduled Integrity Check Test

Description: Verifies that scheduled integrity checks run consistently to maintain data accuracy and system reliability.

I-12 - Data Corruption Handling Test

Description: Ensures that the system can identify and handle data corruption effectively, preserving data integrity.

IM-11 - System Malware Vulnerability Scan Test

Description: Conducts a scan to detect and address vulnerabilities that could expose the system to malware threats.

EU-11 - Verify Within 3 Clicks

Description: Checks that users can access critical features and information within three clicks, enhancing usability.

EU-21 - Test Setup Time

Description: Measures the time required to set up the system, aiming for minimal setup duration and ease of use.

PI-11 - Verify Language Selection

Description: Ensures that users can select their preferred language easily and that the system reflects the chosen language accurately.

LR-11 - Confirm Tutorial Activation

Description: Confirms that tutorials are accessible and activate correctly to guide new users through key features.

UP-11 - Verify System Messages Are Clear

Description: Checks that system messages are clear and comprehensible, improving user understanding and interaction.

AREQ-11 - Test Screen Reader Functionality

Description: Ensures compatibility with screen readers, supporting accessibility for visually impaired users.

ARP-11 - Confirm Color Match to Official Brand Palette

Description: Validates that the color scheme used in the system matches the official brand palette for visual consistency.

ARP-12 - Validate Consistency Across Buttons

Description: Ensures that button designs and styles are consistent throughout the interface for a cohesive user experience.

STY-11 - Confirm Use of Neutral Tone

Description: Verifies that all system messages maintain a neutral tone, avoiding biases and ensuring inclusivity.

STY-21 - Verify Error Message Uses Friendly Tone

Description: Checks that error messages are worded in a friendly manner, reducing user frustration.

RR-11 - Review of Release Schedule Adherence

Description: Ensures that the project adheres to the planned release schedule, maintaining project timeline integrity.

CR-11 - Documented Audit Reports and Legal Approvals

Description: Confirms that audit reports and necessary legal approvals are documented for compliance.

SR-11 - Verification Report Against Industry Standards

Description: Verifies that the system meets relevant industry standards, providing assurance of quality and compliance.

III Design

21 Design Goals

The primary design objectives for the OceanSweep platform are to prioritize usability, scalability, and impact. These objectives ensure that the system effectively mobilizes community efforts to combat ocean and beach waste. Key objectives include:

1. **User-Friendly Interface:** A user-friendly and intuitive interface that enables users of all demographics to engage effortlessly with the platform.
2. **Community Engagement:** Features that foster collaboration, such as event coordination tools, progress tracking, and reward systems.
3. **Scalability:** A robust architecture to accommodate a growing user base and adapt to various geographic locations.
4. **Data-Driven Insights:** Providing actionable analytics to measure the impact of clean-up efforts and inform future campaigns.
5. **Sustainability:** Encouraging eco-friendly practices within the platform's operations and user actions.

22 Current System Design

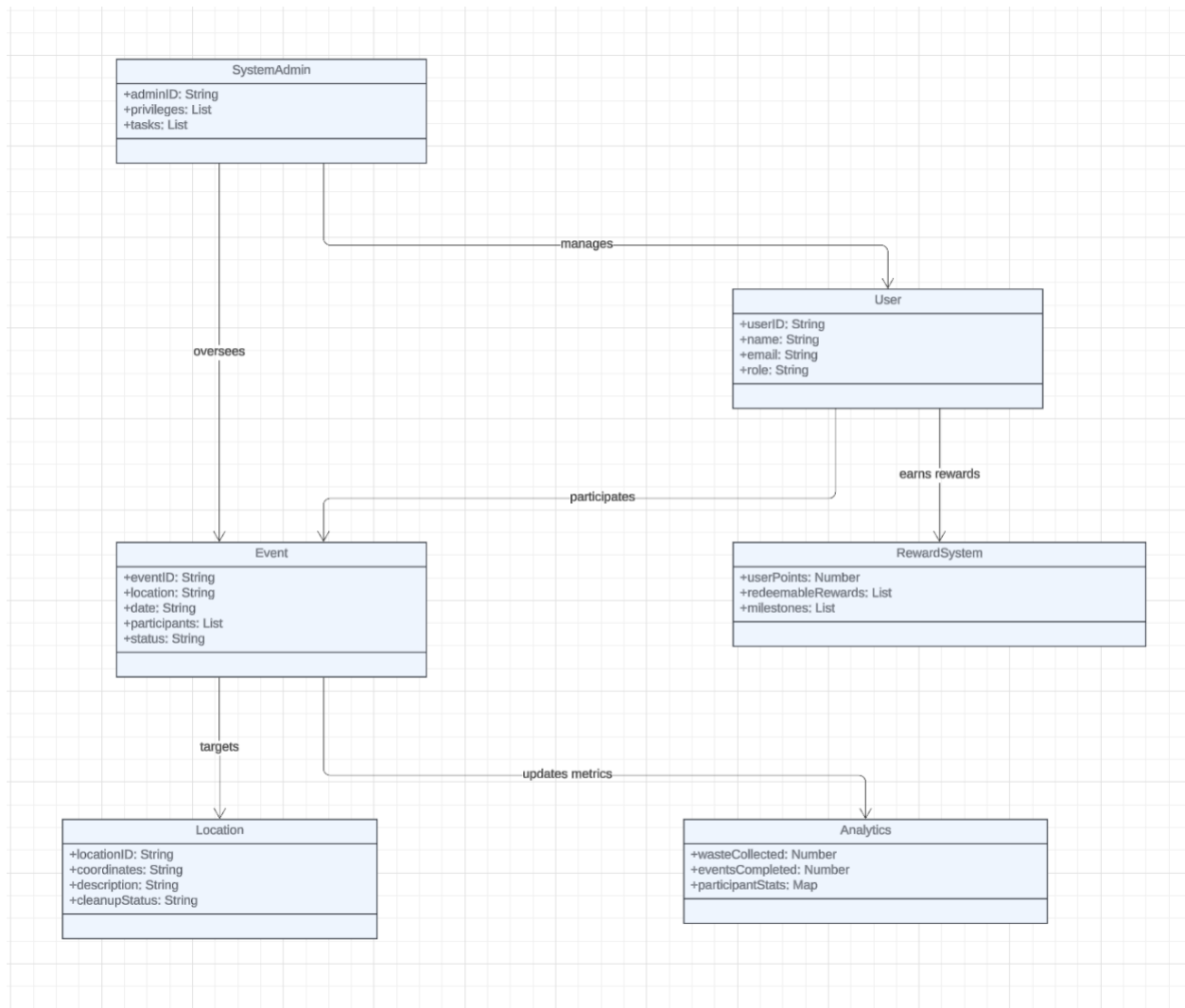
There's no existing system that directly addresses OceanSweep's goals and functionality. The proposed platform is a novel solution designed to fill this gap by creating a dedicated, interactive web application that mobilizes community efforts for ocean and beach waste cleanup.

23 Proposed System Design

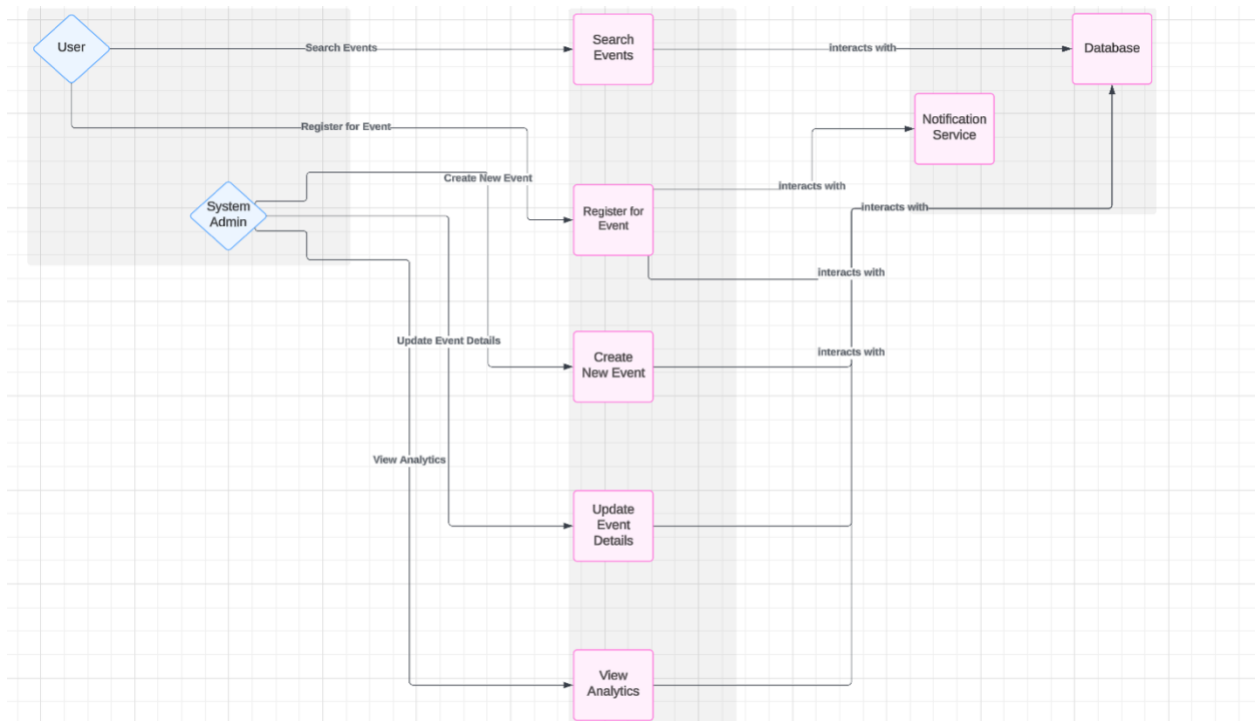
23a Initial System Analysis and Class Identification

The initial system analysis delves into the fundamental classes that underpin the functionality and structure of the OceanSweep platform. By meticulously analyzing the requirements and core functionalities, the following major classes were identified and organized into an initial class structure:

1. **User:** This class represents individual users who interact with the platform. It encompasses attributes such as userID, name, email, and role (e.g., participant, organizer).
2. **Event:** This class encapsulates the details of clean-up events, including eventID, location, date, participants, and status.
3. **Location:** This class stores information about the target areas for clean-up, including locationID, coordinates, description, and cleanupStatus.
4. **Analytics:** This class provides valuable insights into the impact of the platform, storing metrics such as wasteCollected, eventsCompleted, and participantStatistics.
5. **RewardSystem:** This class diligently tracks and manages user rewards for their participation, encompassing attributes like userPoints, redeemableRewards, and milestones.
6. **SystemAdmin:** This class handles administrative functions, including user and event management.



23b Dynamic Modelling of Use-Cases



23c Proposed System Architecture

The OceanSweep platform adopts a **Client-Server Architecture** with elements of the **Model-View-Controller (MVC) design pattern** to ensure scalability, maintainability, and efficient user interactions.

Architecture Description:

1. Client-Server Architecture:

- The system is divided into two primary layers:
 - **Client Layer:** Handles the user interface, allowing users to interact with the platform via a web browser or mobile application.
 - **Server Layer:** Manages business logic, database operations, and communication with external services, such as geolocation APIs and notification systems.

2. Model-View-Controller (MVC) Pattern:

- **Model:** Represents core data entities, such as users, events, locations, and analytics.
- **View:** Manages the presentation layer, dynamically updating the user interface based on data from the server.

- **Controller:** Handles user requests, processes input, and interacts with the Model and View to execute the required operations.

The client-server model enables the system to scale independently on both the client and server sides, effectively handling increased user loads. The MVC pattern's separation of concerns ensures that individual components can be modified or upgraded without significantly impacting the rest of the system. The architecture supports multiple client platforms, such as web and mobile, ensuring OceanSweep's accessibility to a diverse user base. Integration with external APIs, including geolocation services and notification systems, and future enhancements can be achieved with minimal disruption. By offloading complex computations and database operations to the server, the client can concentrate on delivering a seamless and responsive user interface.

23d Initial Subsystem Decomposition

The OceanSweep platform is divided into the following subsystems, each with defined responsibilities and clear boundaries. These subsystems are designed to ensure modularity, scalability, and maintainability.

1. User Management Subsystem

- In our system, two key classes are integral to its functionality: User and RewardSystem. The User class is responsible for managing user authentication, registration, and profile management. It ensures that users can securely create accounts and access their profiles. Meanwhile, the RewardSystem class tracks user rewards, specifically for their participation in clean-up events.

2. Event Management Subsystem

- The system is designed to manage key classes, specifically focusing on events and locations. Its primary responsibilities include facilitating the creation, update, and deletion of clean-up events. In addition, it is tasked with maintaining comprehensive details regarding the event locations, participants involved, and the current statuses of these events.

3. Analytics Subsystem

- The key responsibilities of the Analytics class involve tracking and reporting various metrics. This includes monitoring the amount of waste collected, the number of participants involved, and the events that have been completed. Additionally, the class provides valuable insights to both users and administrators to enhance the overall understanding of these metrics.

4. Notification Subsystem

- The key class associated with this system is the NotificationService. Its primary responsibilities include sending event reminders, registration

confirmations, and various alerts to users, ensuring they stay informed and engaged.

5. Administrative Subsystem

- The key classes within the platform include SystemAdmin and Event. The responsibilities of these classes involve enabling administrators to oversee and manage various aspects of the platform, including events, users, and analytics. This oversight ensures the smooth operation of the overall system.

24 Additional Design Considerations

The following design considerations have been analyzed to ensure the OceanSweep platform meets its goals effectively. These considerations may introduce additional classes or functionality as required.

1. Security and Privacy

- Concentrating on protecting user data, event details, and analytics from unauthorized access.
 - Implement secure user authentication using OAuth 2.0.
 - Encrypt sensitive data at rest and in transit.
 - Introduce a SecurityManager class to enforce role-based access control (RBAC).

2. Scalability

- Concentrating on supporting a growing user base and increased event data volume.
- **Implementation:**
 - Use scalable cloud infrastructure (e.g., AWS or Azure).
 - Introduce a LoadBalancer class to distribute user requests evenly across servers.

3. Usability

- Concentrating on ensuring the platform is intuitive and accessible to users of varying technical expertise.
- **Implementation:**
 - Conduct user testing to refine the interface.
 - Introduce a HelpSystem class to provide guidance and tutorials.

4. Integration with External Services

- Concentrating on relying on third-party APIs for geolocation, notifications, and analytics.
- **Implementation:**
 - Introduce APIHandler and GeoService classes to handle communication with external APIs.
 - Add retry logic for API calls to handle temporary failures.

5. Performance

- Concentrating on ensuring the system responds quickly to user actions.
- **Implementation:**
 - Use caching mechanisms for frequently accessed data (e.g., event lists).
 - Introduce a CacheManager class to manage caching.

6. Error Handling and Recovery

- Concentrating on Handling unexpected errors gracefully without disrupting the user experience.
- **Implementation:**
 - Introduce an ErrorHandler class to log and manage system errors.
 - Provide clear error messages to users and fallback options where possible.

7. Legal and Ethical Considerations

- Concentrating on complying with data protection laws such as GDPR or CCPA.
- **Implementation:**
 - Ensure user consent for data collection and processing.
 - Include a ComplianceManager class to enforce legal requirements.

24a Hardware / Software Mapping

OceanSweep employs a client-server architecture distributed across multiple hardware and software components to ensure scalability, performance, and ease of maintenance. The hardware and software components are mapped as follows:

Hardware Components

1. Client Devices:

- The client devices represent user devices, such as smartphones, tablets, or desktops. Users access the system via a web browser or mobile application.

2. Application Server:

- A cloud-based server that processes requests from the client and also hosts the business logic (back-end) and facilitates communication between subsystems.

3. Database Server:

- A dedicated server that stores data related to users, events, and analytics. Uses a relational database system (e.g., PostgreSQL or MySQL).

4. Notification Server:

- Responsible for sending event reminders, confirmations, and other notifications to users.

5. Third-Party APIs:

- Services such as geolocation APIs for mapping event locations and email APIs for notifications.

Software Components

1. Client Interface:

- Built using web technologies (e.g., React, Angular) or as a mobile app. Which provides an intuitive interface for users to interact with the platform.

2. Business Logic:

- Implemented on the application server to handle core system operations, such as user registration and event management.

3. Database:

- Stores all system data, ensuring fast and efficient retrieval and implements appropriate security measures for data protection.

4. Communication Subsystems:

- REST APIs to facilitate communication between the client and the server and integration with third-party APIs to support geolocation and notifications.

- Our application server and database server are hosted on cloud platforms like AWS or Azure. This way, they can easily scale up and stay up and running even if something goes wrong. We also use a load balancer to spread out the traffic from clients to multiple application server instances. And to keep our data safe, we use HTTPS.

24b Persistent Data Management

The OceanSweep platform requires a robust and efficient mechanism for managing persistent data to ensure that critical information is stored securely and restored seamlessly when the system starts up. This section outlines the classes and subsystems responsible for persistent data management.

Persistent Data Requirements

The system must store and retrieve several key types of data. First, it will handle **user data**, which includes user profiles containing authentication credentials, contact information, and rewards. Additionally, the system will manage **event data**, detailing clean-up events by providing information about their location, participants, and progress status.

Furthermore, the system will store **location data**, which consists of geolocation information for events, including coordinates and descriptive details. It will also maintain **analytics data**, capturing metrics on the amount of waste collected, the number of events completed, and statistics related to participants. Lastly, the system will contain **system metadata**, encompassing logs, notifications, and administrative settings.

Key Classes for Persistent Data

The following classes represent the core entities that manage persistent data:

1. User Class:

- The system encompasses several key attributes, including userID, name, email, role, and reward points. It is designed to store and manage user information and preferences effectively.

2. Event Class:

- The responsibilities include storing and managing user information and preferences. This involves organizing data effectively to ensure user details are easily accessible and preferences are carefully maintained.

3. Location Class:

- The primary responsibility is to maintain geolocation data for event sites. The attributes include locationID, coordinates, description, and cleanupStatus.

4. Analytics Class:

- The system is designed with several key attributes, including the amount of wasteCollected, the number of eventsCompleted, and the statistics of participantStatistics. Its primary responsibilities involve storing and retrieving usage metrics, which are essential for generating comprehensive reports on the system's performance.

5. Notification Class:

- The system is designed to manage notifications with several key attributes, including notificationID, type, recipient, message, and timestamp. It is responsible for persisting the notification history, ensuring that user alerts are recorded and accessible for future reference.

Subsystem for Persistent Data Management

1. Database Subsystem:

- In our technology stack, we utilize a relational database like PostgreSQL or MySQL for structured data storage. Our primary responsibilities include the storage, retrieval, and updating of persistent data for core entities. We also prioritize maintaining data consistency and integrity by implementing transactions effectively.

2. Backup and Restore Subsystem:

- Automatically back up critical data at regular intervals and restore data in case of system failure or restart.

3. Data Access Layer:

- Provides an abstraction for accessing and manipulating data stored in the database and encapsulates SQL queries and ensures a clean interface for interacting with persistent storage.
- Data integrity is maintained through the use of transactions, which ensure consistency in the data during updates. In terms of security, sensitive information, such as user credentials, is encrypted both during transmission and while stored. Additionally, the database schema is designed to accommodate growing data volumes as the platform expands.

24c Access Control and Security

Access control and security are critical components of the OceanSweep system to protect sensitive data, ensure authorized access, and safeguard system operations from

potential threats. This section identifies the security concerns and proposes additional classes and subsystems to address these concerns.

Access Control and Security Concerns

1. Authentication and Authorization:

- Making sure that only authorized users, such as participants, organizers, and administrators, can access certain sections of the system. This involves using role-based access control (RBAC) to allocate permissions according to the user's role.

2. Data Confidentiality:

- Protecting sensitive user data, such as contact information and event details, from unauthorized access.

3. Data Integrity:

- Preventing unauthorized modifications to data, including events, analytics, and user profiles.

4. Secure Communication:

- Encrypting data transmitted between client devices and the server using HTTPS to prevent eavesdropping.

5. System Resilience:

- Protecting against threats like SQL injection, cross-site scripting (XSS), and distributed denial-of-service (DDoS) attacks.

Classes and Subsystems

1. AuthenticationManager Class:

- The responsibilities of this role involve managing the processes for user login and logout, ensuring that users can access and exit the system securely. Additionally, it includes verifying user credentials using standards such as OAuth 2.0 or similar methods. The primary attributes associated with this role are the login token, expiration time, and user role.

2. AuthorizationManager Class:

- The responsibilities of the role include enforcing role-based access control (RBAC) for various parts of the system, ensuring that users have the appropriate level of access. Additionally, the role involves validating user permissions for specific actions, confirming that individuals are authorized to perform those tasks.

3. **EncryptionHandler Class:**

- The responsibilities of the role include enforcing role-based access control (RBAC) for various parts of the system, ensuring that users have the appropriate level of access. Additionally, the role involves validating user permissions for specific actions, confirming that individuals are authorized to perform those tasks.

4. **SecurityManager Subsystem:**

- The responsibilities include monitoring for potential security threats, which involves keeping an eye out for unusual login patterns or unauthorized access attempts. Additionally, this role requires logging and reporting any security incidents that occur.

5. **Firewall and DDoS Protection:**

- The system continually monitors for potential security threats by observing unusual login patterns and unauthorized access attempts. In addition to this proactive approach, it also logs and reports any security incidents that may occur, ensuring that all relevant information is documented for further analysis and response.

Implementation Details

1. **Role-Based Access Control (RBAC):**

- In the system, there are three distinct roles with specific capabilities. The User role allows individuals to view and register for events, providing them with access to event information. The Organizer role is responsible for creating, modifying, and deleting events, ensuring that the event listings are up-to-date and accurately reflect the available offerings. Finally, the Admin role possesses full access to the system, which includes the ability to manage users and oversee analytics. Permissions associated with these roles are systematically mapped and enforced at the application server level, ensuring secure and appropriate access for each user type.

2. **Secure Data Storage:**

- Passwords are securely hashed using a strong algorithm to protect user information. Additionally, sensitive data is stored within encrypted fields in the database, ensuring that it remains safe from unauthorized access.

3. **Communication Security:**

- The HTTPS protocol plays a crucial role in ensuring secure communication between the client and the server. Moreover, when making API calls to external services, authentication is achieved through the use of API keys or OAuth, which helps to protect access to sensitive data and functionalities.

4. Regular Security Audits:

- Regular penetration testing is essential to identify and patch any vulnerabilities that may exist within the system. Additionally, it is important to keep third-party libraries and frameworks updated to address known issues and ensure the overall security of the application.

24d Global Software Control

Global Software Control Concerns

Properly managing software systems requires addressing several overarching concerns to ensure reliability, scalability, and performance. One critical area is **System Initialization and Shutdown**, which involves ensuring the system initializes properly by starting essential subsystems and loading critical data. Equally important is shutting down safely, which includes saving system states and releasing resources appropriately. Another concern is **Error and Exception Handling**, where global errors such as database unavailability or external API failures are managed to minimize disruptions. This involves logging errors and communicating them to administrators or users effectively.

Concurrency Control is another key aspect, requiring mechanisms to manage simultaneous requests from multiple users, preventing conflicts or race conditions. Similarly, **Event Scheduling and Timing** ensures that scheduled tasks, such as sending event reminders or generating analytics reports, are coordinated efficiently. **System Monitoring and Alerts** play a vital role in maintaining the system's health and performance, allowing potential issues to be detected and resolved proactively. Lastly, **Scalability and Load Management** ensures the system can handle increased traffic by distributing the load effectively across multiple servers.

Proposed Classes and Subsystems

To address these concerns, a range of classes and subsystems are proposed:

1. **SystemController Class:** The SystemController is responsible for managing system initialization and shutdown processes. It coordinates the startup of critical subsystems, such as database connections, API integrations, and notification services. Key attributes include `systemStatus`, `initializedComponents`, and `shutdownSignal`.
2. **ErrorHandler Class:** This class captures and logs exceptions occurring at a global level. It provides fallback mechanisms for critical failures, such as retrying failed database operations. Its attributes include `errorType`, `errorMessage`, and `resolutionSteps`.
3. **Scheduler Subsystem:** The Scheduler handles and executes scheduled tasks, such as sending notifications or generating periodic reports, leveraging job schedulers like Celery or Quartz. Its attributes include `taskID`, `taskFrequency`, and `taskStatus`.

4. **MonitoringSubsystem:** This subsystem monitors system performance and resource usage, such as memory, CPU, and database queries. It sends alerts to administrators in case of anomalies. Its attributes include metricType, threshold, and currentValue.
5. **LoadBalancer Subsystem:** The LoadBalancer distributes incoming traffic across multiple application server instances to ensure high availability and prevent overloading a single server.

Implementation Details

1. **System Initialization:** The SystemController ensures that all subsystems are started in the correct sequence. For instance, database connections are initialized before client requests are handled.
2. **Error Logging:** The ErrorHandler logs global exceptions into a centralized logging service, such as the ELK Stack or Sentry, to aid troubleshooting.
3. **Task Scheduling:** The Scheduler Subsystem manages scheduled tasks like reminders or analytics generation using a distributed task queue system.
4. **Real-Time Monitoring:** The MonitoringSubsystem integrates with tools such as Prometheus or Grafana to track the system's health in real time.
5. **Load Balancing:** Load balancing algorithms, such as round-robin or least-connections, are implemented to distribute user requests effectively across servers, ensuring system stability and responsiveness.

24e Boundary Conditions

Boundary Condition Concerns

Effective system management requires addressing various boundary conditions to ensure smooth operations and recovery from unexpected events. During **System Startup**, it is crucial to initialize all subsystems, such as databases, APIs, and notification services, in the correct sequence. This includes loading configuration files and verifying their integrity to prevent potential issues later. Conversely, **System Shutdown** focuses on saving unsaved data to prevent loss and gracefully terminating active processes while releasing resources like database connections and API sessions.

Handling **Abnormal Shutdowns**, such as crashes, power loss, or server outages, requires robust measures to maintain data integrity and ensure the system's availability for recovery. Additionally, **Configuration File Management** involves managing external configuration files containing system settings, such as database credentials and API keys, while ensuring they remain secure and up-to-date.

Database Maintenance includes tasks such as schema migrations, backups, and restoring data after failures. It also emphasizes verifying the database's integrity and maintaining consistency during updates. Lastly, **Log Management** plays a vital role in

maintaining detailed logs of system events, errors, and critical operations, which assist in troubleshooting and audits.

Proposed Classes and Subsystems

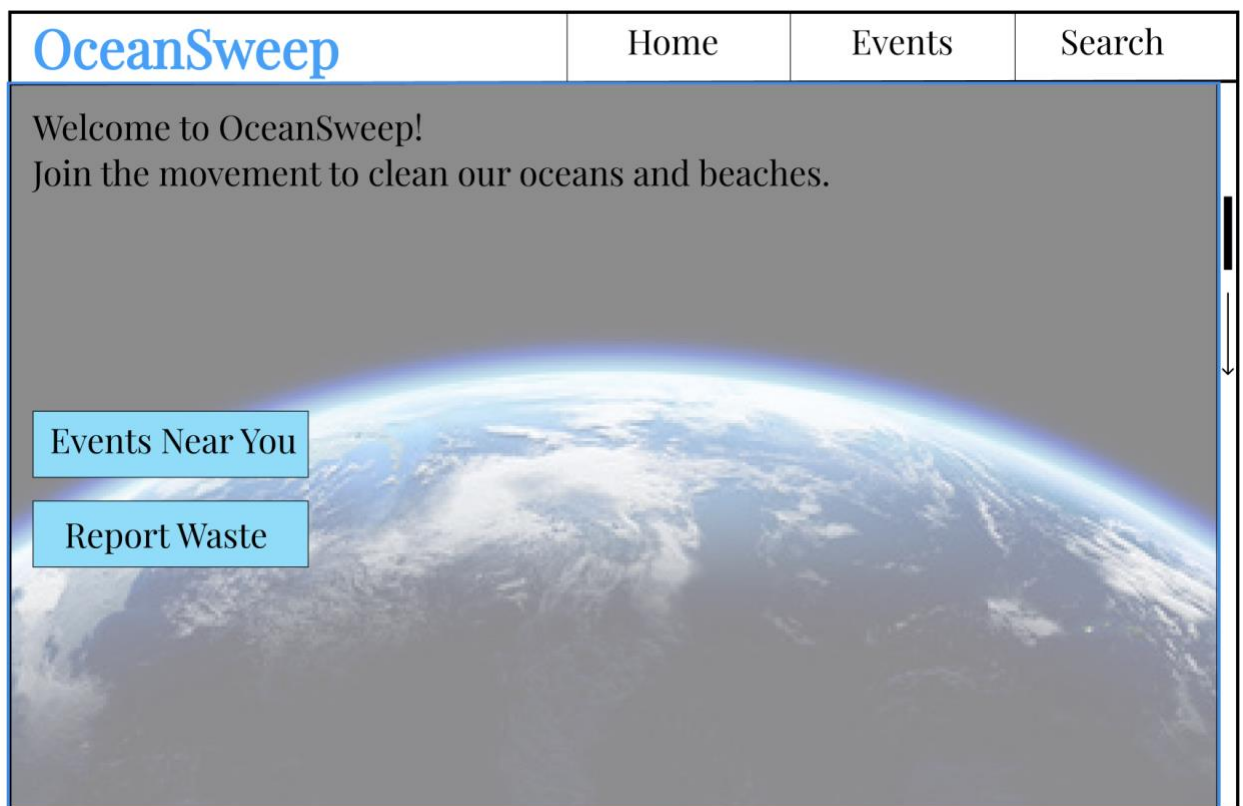
1. **StartupManager Class:** The StartupManager handles the initialization of all critical subsystems in a specific sequence. It also loads configuration files and verifies their correctness. Its key attributes include initializedSubsystems and configStatus.
2. **ShutdownManager Class:** The ShutdownManager ensures a graceful termination of active processes, saves critical data, and releases system resources such as open database connections and API sessions. Its attributes include processStatus and saveState.
3. **RecoveryManager Class:** This class is responsible for recovery operations following abnormal shutdowns, such as restarting services and restoring data. It also validates data consistency and restores corrupted files when needed. Key attributes include recoveryMode, backupSource, and recoveryStatus.
4. **ConfigManager Subsystem:** The ConfigManager handles reading, updating, and securing configuration files, ensuring sensitive settings like database credentials or API keys are encrypted and stored securely. Its attributes include configFilePath, lastUpdated, and secureMode.
5. **Database Maintenance Subsystem:** This subsystem automates database backup and restore processes, handles schema updates, and verifies database integrity. Its attributes include backupSchedule and schemaVersion.
6. **LogManager Class:** The LogManager maintains logs of system startup, shutdown, errors, and recovery events. It provides tools for querying and analyzing logs to facilitate debugging. Its key attributes include logFilePath and logLevel.

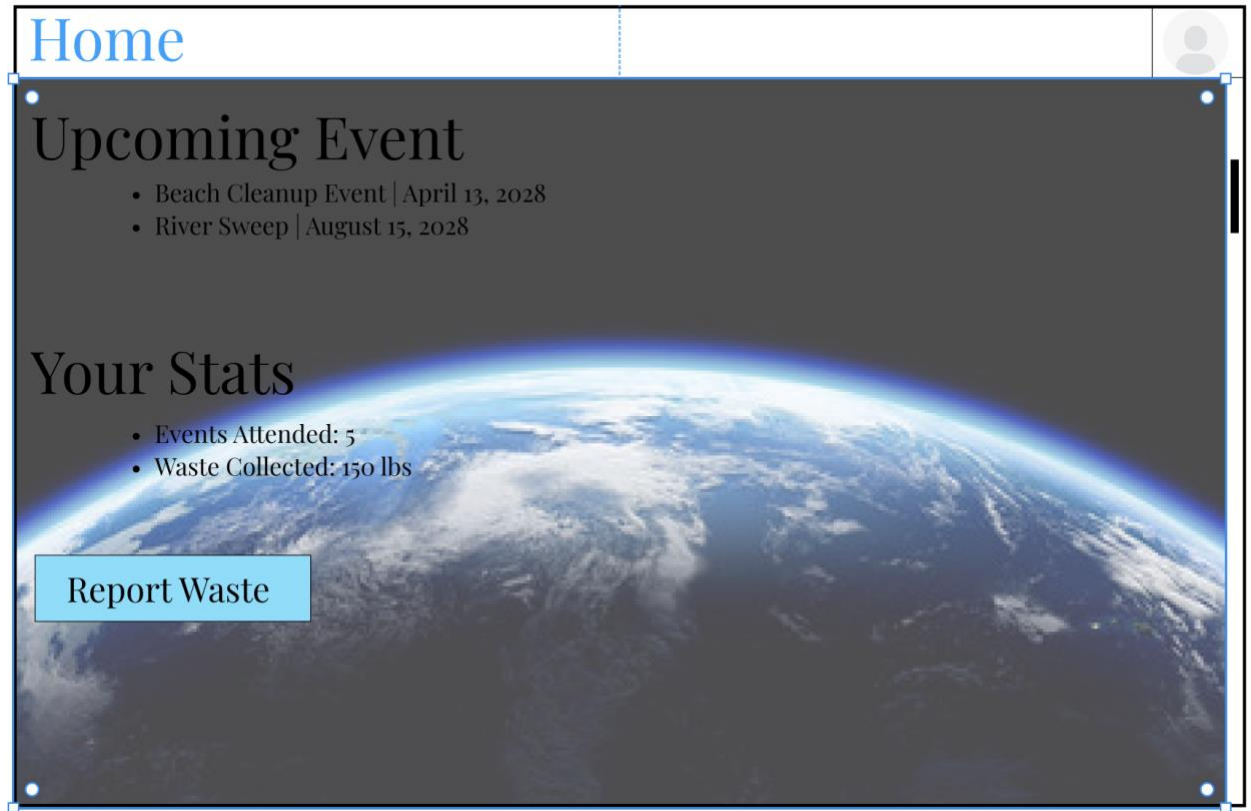
Implementation Details

1. **Startup Sequence:** The StartupManager initializes subsystems in a defined order: loading configuration files, initializing database connections, starting application services, and establishing connections to external APIs.
2. **Graceful Shutdown:** The ShutdownManager ensures all active sessions are terminated, and critical data is saved before the server shuts down, avoiding data loss or resource leaks.
3. **Recovery from Abnormal Shutdown:** The RecoveryManager utilizes backup data and logs to restore the system's state after unexpected crashes, ensuring minimal downtime and data recovery.

4. **Configuration File Security:** Configuration files are encrypted and accessible only to authorized subsystems. Changes to these files are logged for audit purposes, enhancing security and traceability.
5. **Database Backup and Restore:** Scheduled backups ensure data availability, while automated restore processes enable rapid recovery from failures, minimizing disruption.
6. **Logging and Monitoring:** The LogManager logs all critical events, such as startup, shutdown, errors, and recovery operations. These logs are invaluable for debugging and auditing, helping maintain system reliability.

24f User Interface





25 Final System Design

The final system design for the OceanSweep platform integrates all subsystems and classes identified during the design process. This section provides an overview of the complete system architecture, showing the interactions between subsystems and the key classes within each.

Overall Package Diagram

The system is divided into the following subsystems:

1. User Management Subsystem

- Manages user registration, authentication, profiles, and rewards.
- Classes:
 - User
 - AuthenticationManager
 - RewardSystem

2. Event Management Subsystem

- Handles the creation, modification, and registration of clean-up events.
- Classes:
 - Event
 - Location
 - NotificationService

3. Analytics Subsystem

- Provides insights into system performance and user engagement.
- Classes:
 - Analytics
 - ReportGenerator

4. Security and Access Control Subsystem

- Ensures data integrity and controls access based on user roles.
- Classes:
 - SecurityManager
 - AuthorizationManager
 - EncryptionHandler

5. Persistent Data Management Subsystem

- Stores and retrieves data from the database.
- Classes:
 - DatabaseManager
 - BackupHandler

6. System Control Subsystem

- Oversees global operations such as startup, shutdown, and error handling.
- Classes:
 - SystemController

- ErrorHandler

Subsystems and Classes

1. User Management Subsystem:

- Handles all user-related functionality, such as account management and tracking rewards for participation.
- **Classes:**
 - User: Stores user details and role information.
 - AuthenticationManager: Manages login/logout processes and session tracking.
 - RewardSystem: Tracks and updates user rewards.

2. Event Management Subsystem:

- Manages clean-up events, including scheduling, location details, and participant tracking.
- **Classes:**
 - Event: Represents individual events with attributes like date, location, and status.
 - Location: Stores geolocation data and descriptions for events.
 - NotificationService: Sends reminders and confirmations to users.

3. Analytics Subsystem:

- Tracks system performance metrics and generates reports for users and administrators.
- **Classes:**
 - Analytics: Stores data on participation trends and event outcomes.
 - ReportGenerator: Creates visual reports for the dashboard.

4. Security and Access Control Subsystem:

- Manages authentication, encryption, and role-based access control to ensure data security.
- **Classes:**

- `SecurityManager`: Monitors system activity and manages security protocols.
- `AuthorizationManager`: Enforces permissions based on user roles.
- `EncryptionHandler`: Encrypts sensitive data such as passwords.

5. **Persistent Data Management Subsystem:**

- Ensures the persistence of critical data across system restarts and provides backup/recovery functionality.
- **Classes:**
 - `DatabaseManager`: Handles database queries and updates.
 - `BackupHandler`: Automates data backups and restores.

6. **System Control Subsystem:**

- Oversees the initialization, shutdown, and recovery processes for the system.
- **Classes:**
 - `SystemController`: Coordinates system startup and shutdown.
 - `ErrorHandler`: Logs and resolves errors.

26 Object Design

26a Packages

Although OceanSweep is not based on Java, the project utilizes a logical grouping of classes into subsystems, which serve as functional "packages." These groupings improve modularity and maintainability by ensuring related classes are organized together. Below are the primary subsystems and their corresponding logical groupings:

User Management Package

- Handles user-related functionality, such as registration, authentication, and rewards tracking.
- **Classes:**
 - `User`
 - `AuthenticationManager`

- RewardSystem

Event Management Package

- Manages the lifecycle of clean-up events, including scheduling, location details, and participant tracking.
- **Classes:**
 - Event
 - Location
 - NotificationService

Analytics Package

- Tracks system performance and user engagement, generating insights and reports.
- **Classes:**
 - Analytics
 - ReportGenerator

Security and Access Control Package

- Ensures data security and enforces role-based access control for users and administrators.
- **Classes:**
 - SecurityManager
 - AuthorizationManager
 - EncryptionHandler

Persistent Data Management Package

- Ensures that critical system data is securely stored and easily retrievable.
- **Classes:**
 - DatabaseManager
 - BackupHandler

System Control Package

- Manages the overall behavior of the platform, including startup, shutdown, and error handling.
- **Classes:**
 - SystemController
 - ErrorHandler

26b User Management Subsystem

Classes

1. User

- The **User** class represents an individual system user and is responsible for managing user-related information. This includes functionalities such as retrieving and updating the user's profile and managing reward points. The class provides three key public methods: `getProfile()`, which returns the user's profile; `updateProfile(UserProfile profile)`, which allows updates to the user's details; and `addReward(int points)`, which adds reward points to the user's account. There are constraints ensuring that each user has a unique email address and that reward points cannot be negative, maintaining data integrity and a consistent user experience.

2. AuthenticationManager

- The **AuthenticationManager** class is tasked with handling user authentication and session management. This class ensures secure access to the system by providing methods such as `login(String email, String password)`, which authenticates a user and generates a session token; `logout(String token)`, which ends the user's session; and `isAuthenticated(String token)`, which verifies if a session is valid. To ensure security, constraints dictate that passwords must be encrypted and stored securely, and session tokens must expire after a predefined time to prevent unauthorized access.

3. RewardSystem

- The **RewardSystem** class manages the tracking and utilization of user rewards within the system. It includes methods such as `addPoints(int userId, int points)`, which adds points to a user's account; `redeemRewards(int userId, int points)`, which deducts points if a sufficient balance is available; and `getRewardBalance(int userId)`, which retrieves the current reward balance of a user. Constraints are enforced to ensure that reward balances

cannot drop below zero, maintaining consistency and fairness in the rewards program. This class provides a streamlined way to manage user incentives while upholding robust data validation.

26c Event Management Subsystem

Classes

1. Event

- The **Event** class is responsible for representing clean-up events and managing event-related details. It provides key functionalities through its public methods: `createEvent(EventDetails details)`, which creates a new event; `updateEvent(int eventId, EventDetails details)`, which updates the details of an existing event; and `getParticipants(int eventId)`, which retrieves a list of participants for a given event. The class enforces constraints to ensure each event has a unique ID and that event dates cannot be set in the past, maintaining the integrity and relevance of the event data.

2. Location

- The **Location** class handles the geolocation and description associated with an event's venue. Its public methods include `getCoordinates()`, which returns the latitude and longitude of the location, and `updateLocation(int locationId, LocationDetails details)`, which updates the location's information. Constraints ensure that all coordinates provided are valid geographic points, supporting accurate and reliable event mapping.

3. NotificationService

- The **NotificationService** class is responsible for managing notifications related to events. It provides two primary methods: `sendReminder(int eventId)`, which sends reminders to all participants of a given event, and `sendConfirmation(int userId, int eventId)`, which sends a confirmation to a user regarding their participation in an event. The class enforces a constraint ensuring that notifications are sent only to registered users, thereby maintaining relevance and preventing unnecessary communications.

26d Analytics Subsystem

Classes

1. Analytics

- The system is responsible for storing metrics and generating data summaries. It includes two public methods: the `recordMetric(String metricType, int value)` method, which records a new metric, and the

``getMetrics(String metricType)`` method, which retrieves all recorded metrics of the specified type. It is important to note that all metrics must be non-negative.

2. **ReportGenerator**

- The responsibilities involve creating reports based on analytics data. There are two primary public methods available: the first, ``generateParticipationReport()``, generates a report that outlines participation trends, while the second, ``generateWasteReport()``, produces a report detailing the amount of waste collected. It's important to note that all reports must include up-to-date data to ensure accuracy and relevance.

26e Security and Access Control Subsystem

Classes

1. **SecurityManager**

- The primary responsibilities of this role involve monitoring and enforcing security protocols to ensure the safety and integrity of the system. A key aspect of this role is the ability to log and monitor user activities through a method called ``monitorActivity``. This method requires input on the type of activity and additional details, allowing for thorough oversight of user actions. Another essential function is to evaluate the system's security status by using the ``isSecure`` method, which returns a boolean indicating whether the system is secure or not. Additionally, it is important for this role to integrate with external security tools whenever applicable to enhance overall security measures.

2. **AuthorizationManager**

- In the given system, the responsibilities include enforcing role-based access control (RBAC). One of the public methods available is ``checkAccess``, which takes an integer representing the user ID and a string that specifies the resource in question. This method returns a boolean value indicating whether the user has access to that resource. It's important to note that roles must be predefined within the system.

3. **EncryptionHandler**

- The responsibilities of this system involve managing the processes of data encryption and decryption. It provides two public methods: the first, ``encrypt`` is designed to encrypt the input data and return the resulting encrypted string. The second method, ``decrypt``, is meant to take an encrypted string as input and return the original decrypted data. It's essential to note that a secure encryption algorithm, such as AES, must be utilized to ensure the safety and integrity of the data throughout this process.

26f Persistent Data Management Subsystem

Classes

1. DatabaseManager

- The responsibilities of the system include interfacing with the database to effectively store and retrieve data. It provides two key public methods to facilitate its operations: the first method, ``executeQuery``, allows users to execute a database query and returns a ``ResultSet`` containing the results. The second method, ``backupDatabase``, is designed to create a backup of the database by specifying a backup path; it returns a boolean value to indicate the success of the operation. Moreover, it is essential that the database adheres to ACID compliance standards, ensuring reliable transactions and data integrity.

2. BackupHandler

- In this role, responsibilities include managing automated backups to ensure data integrity and security. One of the key public methods available is ``scheduleBackup``, which allows for the scheduling of regular backups. Additionally, the method ``restoreBackup`` provides the functionality to restore the database from a backup using a provided backup path. It is essential to note that all backups must be stored securely to protect sensitive information.

26g System Control Subsystem

Classes

1. SystemController

- The main responsibilities of the role include overseeing both the startup and shutdown processes, as well as the overall global operations of the system. There are two key public methods associated with this position: the first is the ``initializeSystem()`` method, which is responsible for starting the system and initializing its subsystems, while the second is the ``shutdownSystem()`` method, designed to safely shut down the system when necessary. An important constraint to keep in mind is that the subsystems must be initialized in the correct order to ensure proper functionality.

2. ErrorHandler

- The primary responsibilities of the system include logging and managing global errors. It features a public method called ``logError``, which takes a string representing the error message and logs it appropriately. Additionally, there is a method named ``resolveError`` that accepts a string indicating the error code and attempts to resolve the specific error. It is

crucial that all logs remain accessible to facilitate troubleshooting processes.

III Project Issues

27 Open Issues

The following issues have been identified during the design and development phases of the OceanSweep project. These issues remain unresolved at this stage and will require further investigation and decision-making to ensure a complete and functional system.

1. Geolocation Accuracy

- The system relies on external geolocation APIs to map event locations. However, the accuracy of these APIs can vary depending on the provider and user input. Inaccurate locations could potentially impact event organization and user participation. Therefore, evaluating alternative geolocation APIs or implementing a fallback mechanism to verify location data is advisable.

2. Notification Reliability

- User notifications, such as event reminders and confirmations, are contingent upon third-party services that may experience interruptions or delays. Incidental or delayed notifications could result in diminished participation or user discontent. Implement retry mechanisms and diligently monitor the reliability of the notification service.

3. Scalability of Analytics Subsystem

- As the number of events and participants increases, the analytics subsystem may encounter performance issues when processing substantial datasets. This can result in delayed report generation and reduced responsiveness in the dashboard. To address these challenges, consider implementing optimization techniques such as caching, indexing, or transitioning to a more scalable database solution.

4. Role-Based Access Control (RBAC) Implementation

- The RBAC system is conceptually designed but not fully implemented, raising concerns about edge cases such as overlapping roles and temporary permissions. This can lead to security vulnerabilities or user access issues. To address these concerns, it is crucial to develop a comprehensive RBAC policy and thoroughly test it for edge cases during the implementation process.

5. Error Handling for External APIs

- The system heavily relies on external APIs for geolocation, notifications, and analytics. However, the error handling for API failures is not well-defined. API

failures could result in system crashes or reduced functionality. Therefore, it's crucial to define and implement robust error-handling strategies, including fallback mechanisms and user notifications.

6. Cross-Platform Compatibility

- The user interface (UI) design has not been fully tested across all platforms and devices, potentially resulting in usability issues on specific screen sizes or operating systems. This could lead to poor user experience on non-optimized devices. Conduct comprehensive testing across multiple devices and platforms to identify and resolve compatibility issues.

7. Backup and Recovery Testing

- The backup and recovery mechanisms for the database are designed but not yet tested in real-world scenarios. This means that there is a risk of data loss or extended downtime in the event of a failure. To mitigate this risk, we should perform rigorous testing of the backup and recovery processes under different failure scenarios.

8. Data Privacy Compliance

- The system must comply with data privacy regulations (e.g., GDPR, CCPA), but detailed policies and implementations are still under review. Legal and reputational risks if user data is not handled properly. Consult legal experts and update the system to align with privacy laws.

28 Off-the-Shelf Solutions

28a Ready-Made Products

Google Maps API

- Provides geolocation, mapping, and route optimization services, which are crucial for reporting waste locations and organizing events. It is known for its high reliability, global coverage, and robust developer support. However, licensing fees increase with higher usage, necessitating careful budget planning. Alternatively, OpenStreetMap, a free and open-source mapping service, can be cost-effective for budget-conscious implementations.

SendGrid

- Facilitates email notifications for account activation, event reminders, and updates. This scalable and reliable email delivery system provides detailed analytics. While the free tier is sufficient for initial use, paid plans may be required as the user base expands.

Auth0

- Firebase Authentication offers user authentication, including multi-factor authentication (MFA) and social logins. It provides enterprise-grade security and seamless integration with MERN stack applications. However, costs may increase with advanced features or a growing user base. For simpler use cases, Firebase Authentication is a suitable option.

28b Reusable Components

React and Material-UI (MUI)

- Build responsive and user-friendly front-end interfaces. Pre-built, customizable components reduce development time while maintaining professional design standards. May require integration with custom components for branding consistency.

MongoDB

- Cloud-based NoSQL database for managing user data, reports, and event details. Highly scalable and suitable for real-time data requirements. Requires cost management for high data volume.

Chart.js

- JavaScript library for creating interactive data visualizations. Lightweight and compatible with React for rendering charts and graphs. Limited customization options compared to more complex libraries.

28c Products That Can Be Copied

Internal Volunteer Management System

- Developed for a previous project within the same organization to manage volunteer data and schedules. Adapt and integrate its functionality into OceanSweep's event coordination module. Saves development time and leverages existing, tested solutions. Ensure no legal or proprietary restrictions on reuse.

Past Data Backup Scripts

- Scripts from an earlier project handling data backup and recovery processes. Provide a foundation for the backup and restore module in OceanSweep. Proven reliability and familiarity with the development team. Update and optimize the scripts to align with current system architecture.

29 New Problems

29a Effects on the Current Environment

The introduction of the OceanSweep platform has the potential to significantly disrupt established workflows within environmental organizations and among volunteers. For instance, the manual processes currently employed for coordinating cleanup events may need to be replaced by this digital system, necessitating adaptation and training for users. Resistance to change among individuals accustomed to traditional methodologies could hinder the adoption process. Additionally, an over-reliance on the platform may lead to challenges in the event of system downtime or technical failures.

29b Effects on the Installed Systems

The integration of the OceanSweep platform with existing systems can lead to several compatibility issues. For example, when using mapping APIs like Google Maps, conflicts may arise with older systems that rely on different geolocation standards. This can result in discrepancies and affect the accuracy of location-based services. Additionally, incorporating email delivery services such as SendGrid could lead to problems like duplicate notifications unless there's careful coordination with current communication tools. Furthermore, the OceanSweep platform's high resource consumption—such as increased bandwidth or server load—may negatively impact other applications that share the same infrastructure, potentially causing performance bottlenecks.

29c Potential User Problems

Users may face several challenges when adopting the new system:

- **Learning Curve:** Individuals who are not familiar with digital platforms might struggle to navigate the system, especially in remote or underprivileged areas.
- **Trust Issues:** Concerns regarding data privacy and security—particularly among volunteers and environmental organizations—could diminish users' confidence in the platform.
- **Accessibility:** Limited internet connectivity or the use of older devices may hinder some users from fully accessing the platform's features.
- **Overload:** An excessive number of notifications or a complex interface might overwhelm users, leading to reduced engagement and satisfaction.

29d Limitations in the Anticipated Implementation Environment That May Inhibit the New Product

Several physical and technical limitations may impact the product's performance:

- **Unreliable Internet:** Users in remote coastal regions may experience difficulties accessing the platform due to poor connectivity.
- **Device Limitations:** Volunteers and community members with older or less capable devices might face challenges with the platform's performance.
- **Environmental Conditions:** Using the platform in harsh environments, such as beaches with excessive sunlight or moisture, could hinder device usability. Therefore, it is important for the platform to support offline functionality.
- **Power Reliability:** Areas with inconsistent electricity supply could affect the ability to use and recharge devices needed to access the platform.

29e Follow-Up Problems

- **Maintenance Costs:** Ongoing maintenance, including server expenses and software updates, could strain the budgets of non-profit organizations using the platform.
- **Data Overload:** As the platform grows, managing and analyzing large volumes of data (such as trash reports and event participation) may become challenging.
- **Scalability Issues:** Sudden spikes in user activity, particularly during global cleanup events, could overload the system and lead to performance degradation.
- **Regulatory Challenges:** Keeping up with new data privacy regulations, such as GDPR or CCPA, may require frequent updates and audits, which could potentially delay other development efforts.
- **Volunteer Retention:** Users may lose interest over time if recognition or rewards systems do not effectively engage them, which could impact long-term participation rates.

30 Migration to the New Product

30a Requirements for Migration to the New Product

Parallel System Operation: The existing system and the new OceanSweep platform must operate concurrently for at least three months. This duration allows stakeholders to verify the reliability of the new system while giving users adequate time to transition.

User Training: Training sessions must be conducted to ensure that all users, including volunteers, coordinators, and other stakeholders, understand how to use the new platform effectively.

Data Backup: A comprehensive backup of all data from the existing system must be created before the migration begins. This step is essential to prevent any data loss during the transition process.

Data Integrity Checks: After the migration, checks are required to confirm that all data has been successfully transferred and that it is accurate within the new system.

Support Availability: Dedicated technical support must be available throughout the migration process to promptly resolve any issues that may arise.

Roll-Back Plan: A contingency plan should be in place to revert to the old system if any critical issues occur during the migration process.

30b Data That Has to Be Modified or Translated for the New System

User Profiles: Existing user profiles, including names, contact details, roles, and participation history, must be reformatted to conform to the database schema of the new system.

Event Records: Historical data pertaining to past cleanup events must be updated to include additional fields such as geolocation data, participant details, and outcomes, as per the requirements of the new platform.

Trash Reports: Existing waste reports must be modified to include latitude and longitude coordinates, along with any missing metadata necessary for seamless integration with the new mapping API.

Volunteer Logs: Previous volunteer participation logs may require translation to align with the new system's tracking format, ensuring that all contributions are accurately recorded.

Educational Resources: Any educational materials stored in the old system should be reviewed and reformatted to ensure compatibility with the OceanSweep platform's content delivery framework.

Notification Templates: Predefined templates for notifications and reminders must be adapted to align with the new system's communication architecture.

31 Risks

Every project carries inherent risks that could lead to failure, underperformance, or delays. For OceanSweep, the following risks have been identified, along with their probabilities and potential impacts:

1. Inaccurate Metrics

- Metrics for user engagement, trash collection, or event participation may often be inaccurately captured or misrepresented as a result of poor data collection or processing methods. The impact of these inaccuracies can be significant, leading to

misinformed decision-making and a reduction in stakeholder trust. To mitigate these issues, it is essential to implement robust validation checks and automated data analysis tools to ensure the accuracy of the metrics. Additionally, conducting regular reviews and audits of the data can further help maintain its integrity and reliability.

2. Excessive Schedule Pressure

- Unrealistic deadlines can lead developers to compromise on quality or overlook essential testing phases. This often results in a buggy, incomplete, or unstable platform that ultimately fails to meet user expectations. To mitigate these issues, it's crucial to establish realistic timelines that take into account resource availability and the project's scope. Additionally, incorporating buffer periods into the project plan can help accommodate unforeseen delays, ensuring a more reliable development process.

3. Creeping User Requirements

- During the development process, stakeholders or users may request additional features, which can result in scope creep. This situation not only overwhelms the development team but also drives up costs and delays delivery. To mitigate these challenges, it is essential to implement a formal change management process that assesses and approves any new requirements. Additionally, clearly communicating the project scope and deadlines to stakeholders helps ensure that everyone remains aligned and focused on the initial objectives.

4. Inadequate Measurement

- Failing to adequately measure performance, identify bugs, or assess user satisfaction may lead to unresolved issues and missed opportunities for improvement. This can result in decreased effectiveness of the platform and lower user engagement. To address these challenges, it is important to establish clear performance metrics and implement monitoring tools that will track progress toward goals. Additionally, creating continuous feedback loops from users will help gather their insights, allowing for ongoing enhancements.

5. Inaccurate Cost Estimating

- Underestimating the costs associated with development, hosting, or scaling the platform can put significant strain on the budget and may necessitate cutbacks on the project. This miscalculation can lead to delays or forced compromises on essential features, which could ultimately affect the overall success of the platform. To mitigate these risks, it is crucial to conduct a thorough cost analysis that incorporates insights from both financial and technical experts. Additionally, maintaining contingency funds for unexpected expenses can help navigate unforeseen challenges, ensuring that the project remains on track.

6. Low Productivity

- The productivity of the development team may decline for various reasons, including burnout, unclear requirements, and technical challenges. This decline can lead to slower progress, reduced quality of work, and missed deadlines. To mitigate these issues, it is essential to establish clear objectives, provide team members with the necessary tools and resources, and foster a healthy work-life balance.

7. Cancelled Project

- The project faces the risk of cancellation due to several factors, including funding shortfalls, a lack of stakeholder buy-in, and unresolved issues. If such a cancellation occurs, it would result in a complete loss of the time and resources that have been invested. To mitigate these risks, it is essential to secure funding early in the project and maintain stakeholder engagement through regular progress updates. Additionally, developing a minimum viable product (MVP) can help demonstrate the project's value early in the process, thereby bolstering support and interest among stakeholders.

8. Silver Bullet Syndrome

- Over-reliance on a single tool, library, or method to address various challenges can ultimately lead to failure if that particular solution proves inadequate. This dependency often results in increased technical debt and limits flexibility within the system. To mitigate these issues, it is crucial to embrace a diverse range of tools and approaches that are specifically suited to the challenges at hand. Additionally, ensuring that the system design is modular will allow for the easy replacement of components as needed, fostering adaptability and reducing the risk of being trapped by a single solution.

9. Low Quality

- Insufficient testing or rushed development can lead to a poorly designed product filled with bugs and usability issues. This can result in a negative user experience, ultimately leading to reduced adoption of the product and the potential need for costly rework. To mitigate these risks, it is essential to prioritize quality assurance through thorough testing and regular reviews. Additionally, allocating sufficient time for debugging and refining the platform will help ensure a more successful outcome.

10. Management Malpractice

- Ineffective leadership, a lack of clear communication, and misaligned priorities can significantly derail a project. These issues often lead to team conflicts, mismanagement of resources, and ultimately, delayed progress. To mitigate these risks, it is essential to define clear roles and responsibilities for each team member. Maintaining open lines of communication is crucial, as is involving all team members in the decision-making processes. By implementing these strategies, the project can be steered toward success while minimizing potential obstacles.

32 Costs

The cost estimate for the OceanSweep project considers monetary costs, time, resource allocation, and potential lost opportunities. Below is a detailed breakdown based on measurable deliverables and anticipated project phases:

Monetary Costs

1. Development Team Salaries:

- *Developers (4):* \$100/hour x 500 hours = \$200,000
- *UI/UX Designers (2):* \$80/hour x 300 hours = \$48,000
- *Project Manager (1):* \$120/hour x 200 hours = \$24,000
- *Total Personnel Costs: \$272,000*

2. Technology and Infrastructure:

- Cloud Hosting (AWS, GCP, or Azure): \$1,500/month x 12 months = \$18,000
- API Licenses (e.g., Google Maps API): \$5,000/year
- Development Tools and Software Licenses: \$10,000
- Total Technology Costs: **\$33,000**

3. Training and Support:

- User Training Sessions: \$5,000
- Technical Support for Migration: \$3,000
- Total Training and Support Costs: **\$8,000**

4. Marketing and Outreach:

- Social Media Campaigns: \$7,000
- Partner Engagement and Volunteer Recruitment: \$5,000
- Total Marketing Costs: **\$12,000**

5. Contingency Fund:

- 10% of Total Costs: \$32,500

Estimated Total Monetary Costs: \$357,500

Time Costs

1. Development Timeline:

- Estimated Completion Time: 9 months (full-time effort by the team)
- Parallel Testing Period: 3 months

2. User Transition Period:

- Estimated 2 months for full user adoption and system stabilization.

3. Opportunity Costs:

- Potential delays in focusing on other environmental initiatives during development.

Resources

1. Personnel Requirements:

- 4 Full-Stack Developers (MERN Stack expertise)
- 2 UI/UX Designers
- 1 Project Manager

2. Infrastructure:

- Cloud-hosted servers capable of supporting 1,000 concurrent users.
- Data storage capacity for 1 TB of information.

3. Third-Party Tools and APIs:

- Google Maps API for geolocation and mapping.
- Firebase for authentication and notification management.

Cost Ranges

Given the early stages of the project, the cost estimate may vary based on unexpected requirements changes or integration challenges.

- **Minimum Estimate:** \$300,000
- **Maximum Estimate:** \$400,000

This range ensures a realistic understanding of costs while accommodating unforeseen developments.

The OceanSweep project is estimated to cost approximately **\$357,500**, including all development, infrastructure, training, and marketing expenses. This estimate provides a solid foundation for resource allocation and project planning, ensuring the client is well-informed of the expected investment.

33 Waiting Room

The waiting room serves as a repository for features and requirements that are not part of the current release of OceanSweep but hold potential for inclusion in future iterations. Below is a list of such requirements, categorized and prioritized based on potential benefit and feasibility:

Future Requirements and Features

1. Enhanced Analytics Dashboard

- Integration of advanced analytics tools to provide predictive insights about pollution trends and cleanup effectiveness. To enable organizations and coordinators to make data-driven resource allocation and planning decisions.

2. Gamification Features

- Implementation of leaderboards, achievements, and gamified incentives to increase user engagement. To motivate volunteers and foster community participation by rewarding environmental contributions.

3. Mobile App Development

- Development of native iOS and Android applications for a smoother mobile experience and offline functionality. To cater to the growing user base that prefers mobile devices for reporting and participation.

4. AI-Based Waste Identification

- Use of AI to analyze uploaded waste images for automated classification and volume estimation. To reduce manual input and improve the accuracy of waste reports.

5. Real-Time Collaboration Tools

- Integration of chat or messaging features for volunteers and coordinators to communicate during cleanup events. To enhance coordination and foster teamwork during events.

6. Expanded Educational Resources

- Addition of interactive quizzes, certifications, and in-depth modules about marine conservation. To raise awareness and build knowledge among users about pollution and its impact on marine ecosystems.

7. **Integration with Local Waste Authorities**

- Direct reporting to and coordination with municipal waste authorities for efficient trash disposal. To streamline the waste removal process and reduce delays in cleanup operations.

8. **Multilingual Support**

- Expanding language options to include regional and international languages beyond the current scope. To make the platform accessible to a global audience.

34 Ideas for Solutions

This section captures potential ideas for implementation and testing that have emerged during the requirements and design phases of OceanSweep. These ideas are not prescriptive but serve as suggestions to guide developers and testers in the later stages of the project.

Implementation Suggestions

1. **Technology Stack**

- *Use of MERN Stack:* Leverage MongoDB, Express.js, React.js, and Node.js for robust full-stack development. This stack provides scalability, flexibility, and a modern ecosystem.

2. **Integrated Development Environment (IDE)**

- *Recommendation:* Use Visual Studio Code for its extensibility, debugging tools, and built-in support for JavaScript and Node.js.

3. **Third-Party APIs**

- *Mapping API:* Incorporate OpenStreetMap or Google Maps API for geolocation and visualization of cleanup events.
- *Email and Notifications:* Use services like SendGrid for email notifications and Firebase Cloud Messaging for real-time updates.

4. **Libraries and Frameworks**

- *Frontend Framework:* Utilize Material-UI or Tailwind CSS for consistent and responsive UI components.

- *Backend Framework:* Employ Express.js for its lightweight and flexible server-side architecture.
- *State Management:* Use Redux or Context API for efficient state management in the frontend.

5. Database Management

- *Database Schema:* Design the database using a NoSQL model (MongoDB) for better handling of unstructured data and scalability.

Testing Suggestions

1. Automated Testing

- Use Jest and Mocha for unit testing of both frontend and backend components.
- Implement Cypress for end-to-end testing to ensure smooth user interactions.

2. Load Testing

- Employ tools like Apache JMeter to simulate heavy user activity and test system performance under stress.

3. Security Testing

- Perform vulnerability scans using tools like OWASP ZAP to identify potential security issues.
- Conduct penetration testing to simulate cyber-attacks and ensure data protection.

4. Usability Testing

- Organize user testing sessions to gather feedback on interface design and workflow usability.
- Incorporate analytics tools like Hotjar to understand user behavior on the platform.

Future Considerations

1. Cloud Integration

- Deploy the platform on cloud services like AWS or Google Cloud for reliability and scalability.

- Use Docker containers for consistent development and deployment environments.

2. AI and Machine Learning

- Explore AI-based features such as automated waste classification or predictive analytics for cleanup efficiency.

3. Mobile Compatibility

- Ensure the platform is mobile-responsive and plan for future development of dedicated mobile applications.

4. Open Source Contributions

- Consider open-sourcing some components to attract contributions from the global developer community, fostering innovation and collaboration.

35 Project Retrospective

At the conclusion of the OceanSweep project, a thorough reflection on the process revealed several key insights into what worked well and what could be improved in the future. These reflections serve as a guide for enhancing future projects by building on successes and learning from challenges.

What Worked Well

1. Team Collaboration

- The team maintained clear and open communication channels, which facilitated efficient problem-solving and decision-making.
- Regular meetings and status updates ensured that everyone was aligned with the project goals and timelines.

2. Technology Choices

- The decision to use the MERN stack proved highly effective due to its scalability and the seamless integration between components.
- Leveraging third-party APIs like OpenStreetMap for geolocation and SendGrid for email notifications streamlined development.

3. Agile Methodology

- The use of Agile practices, such as iterative development and sprint planning, allowed the team to adapt quickly to changing requirements.

4. Documentation and Planning

- Comprehensive documentation throughout the project lifecycle ensured that all team members were informed and that future developers could easily onboard.

What Did Not Work Well

1. Initial Time Estimation

- Underestimating the time required for certain development tasks led to stress during sprints. Future projects should allocate buffer time for unforeseen challenges.

2. Over-Reliance on Third-Party Tools

- While third-party APIs added functionality, occasional downtimes and API limitations disrupted progress. A fallback strategy for critical third-party services should be planned.

3. Security Oversights in Early Phases

- Security considerations were not prioritized during the initial design stages, leading to additional work during implementation to address vulnerabilities.

Recommendations for Improvement

1. Enhanced Time Management

- Use historical data from this project to improve time estimation for similar tasks in the future.

2. Prioritize Security Early

- Adopt a "security-first" approach by conducting threat modeling during the design phase.

3. Backup Strategies for External Dependencies

- Develop contingency plans for critical third-party services to minimize disruptions.

IV Glossary

A

- **API (Application Programming Interface):** A set of protocols and tools that allows software applications to communicate with one another. OceanSweep uses APIs for integrating geolocation and email notification services.

B

- **Boundary Conditions:** Specific conditions the system must handle, including startup, shutdown, and error states.
- **Backend:** The server-side part of the application responsible for processing logic, database interactions, and communication with the client-side.

C

- **Client:** The user's device (e.g., smartphone or computer) that interacts with the OceanSweep platform.

D

- **Dashboard:** A personalized interface that provides users with an overview of their activities, events, and other relevant information.
- **Data Integrity:** The accuracy and consistency of data during its lifecycle, particularly important during migrations or updates.

E

- **Error Handling:** Mechanisms implemented to detect, manage, and resolve errors within the system.

F

- **Frontend:** The client-side part of the application, including the user interface and visual elements users interact with.

G

- **Geolocation:** The identification of a user's physical location using GPS or similar technologies.

M

- **Migration:** The process of transferring data from an existing system to the new OceanSweep platform.

N

- **Notification System:** A feature of the OceanSweep platform responsible for sending alerts, reminders, and updates to users via email or in-app messages.

P

- **Persistent Data:** Information that is saved and remains accessible even after the system is shut down, such as user profiles and event records.
- **Prototype:** An early version of the system used to test concepts and functionalities.

R

- **Role-Based Access Control (RBAC):** A security model used to restrict system access based on user roles, ensuring appropriate permissions are applied.

S

- **Subsystem:** A smaller, self-contained system that is part of the larger OceanSweep application, such as the reporting or notification subsystem.
- **Stakeholders:** Individuals or groups with an interest in the OceanSweep project, including volunteers, coordinators, and administrators.

U

- **User Interface (UI):** The visual and interactive elements of the OceanSweep platform that users engage with to perform tasks.
- **Usability Testing:** The process of evaluating the user interface by testing it with real users to ensure it meets their needs effectively.

V

- **Volunteer Logs:** Records of individual volunteer contributions, including event participation and collected waste data.

V References / Bibliography

Administrations, National Oceanic and Atmospheric. *Home Page*. 10 November 2024. <www.noaa.gov>.

Bell, John. *Underwater Archaeological Survey Report Template: A Sample Document for Generating Consistent Professional Reports*. Chicago: Underwater Archaeological Society of Chicago, 2012.

Fowler, Martin. *UML Distilled, Third Edition*. Boston: Pearson Education, 2004.

Robertson and Robertson. *Mastering the Requirements Process*. n.d.

Silberschatz, Abraham, Peter Baer Galvin and Greg Gagne. *Operating System Concepts*. Ninth. Wiley, 2013.

VI Index

API	Geolocation
21, 22, 23, 91, 97, 99, 111, 115, 118, 119,	110, 125
121, 122, 124, 125	Migration
Backend	6, 114, 118, 125
122, 125	Notification System
Boundary Conditions	7, 33, 35, 43, 126
5, 98, 125	Persistent Data 5, 93, 94, 102, 104, 105, 109,
Client	126
9, 17, 21, 89, 91, 92, 93, 95, 97, 98, 120,	Prototype
125	126
Dashboard	Requirements
.... 43, 47, 63, 64, 68, 70, 75, 78, 120, 125	43
Data Integrity	Role-Based Access Control 56, 57, 65, 72,
..... 51, 57, 64, 71, 79, 95, 115, 125	82, 96, 110, 126
Design	Stakeholders
85	3, 17, 19, 126
Error handling	Subsystem 5, 6, 89, 90, 94, 96, 98, 99, 101,
102, 106, 111	102, 103, 104, 107, 108, 109, 110, 126
Frontend	Usability Testing
122, 125	19, 122, 126
	User Interface
	5, 60, 100, 126
	Volunteer Logs
	115, 126